Universidad de Extremadura

Escuela Politécnica

Ingeniería Informática en Ingeniería del Software

Trabajo Fin de Grado

Desarrollo de una herramienta para la automatización de tareas de reconocimiento en auditorías de seguridad

David Gordillo Burrero

Julio, 2025

Universidad de Extremadura

Escuela Politécnica

Ingeniería Informática en Ingeniería del Software

Trabajo Fin de Grado

Desarrollo de una herramienta para la automatización de tareas de reconocimiento en auditorías de seguridad

Autor: David Gordillo Burrero

Tutor: Andrés Caro Lindo

Resumen

En el presente trabajo se ha desarrollado una herramienta orientada a la automatización de tareas fundamentales en auditorías de seguridad, con un enfoque específico en la fase de reconocimiento. Esta etapa, crítica en cualquier proceso de pentesting, abarca actividades como la recolección de información pasiva y activa, el escaneo de puertos y servicios, la enumeración de subdominios y tecnologías, así como la identificación preliminar de posibles vulnerabilidades.

La solución propuesta integra múltiples utilidades y técnicas ampliamente utilizadas en entornos profesionales de ciberseguridad (como Nmap, Feroxbuster, WhatWeb o WPScan), y se construye sobre una arquitectura modular y escalable que permite incorporar nuevas funcionalidades de forma sencilla en el futuro. A través de una interfaz web intuitiva, el usuario puede gestionar proyectos, ejecutar escaneos personalizados y consultar resultados de manera clara y estructurada.

Esta herramienta permite optimizar el flujo de trabajo durante las fases iniciales de una auditoría, automatizando tareas repetitivas, centralizando la información obtenida y reduciendo significativamente los tiempos de ejecución. Por otra parte, incorpora un sistema de inteligencia artificial basado en modelos generativos, que facilita la interacción con el contexto del escaneo para obtener recomendaciones específicas o definir próximos pasos según las debilidades detectadas. Además, genera automáticamente informes técnicos versionados, lo que simplifica la documentación del proceso y mejora su trazabilidad sin necesidad de intervención manual.

Palabras clave: Seguridad, Hacking Ético, Auditoría, Automatización, Herramienta.

Abstract

This project presents the development of a tool aimed at automating fundamental tasks in security audits, with a specific focus on the reconnaissance phase. This stage, which is critical in any penetration testing process, includes activities such as passive and active information gathering, port and service scanning, subdomain and technology enumeration, and the preliminary identification of potential vulnerabilities.

The proposed solution integrates several tools and techniques widely used in professional cybersecurity environments (such as Nmap, Feroxbuster, WhatWeb, or WPScan), and is built on a modular and scalable architecture that facilitates the future incorporation of new functionalities. Through an intuitive web interface, users can efficiently manage projects, execute customized scans, and review results in a clear and structured way.

The tool helps optimize workflow during the initial stages of a security audit by automating repetitive tasks, centralizing collected data, and significantly reducing execution time. Moreover, it includes an artificial intelligence system based on generative models, which allows users to interact with the scan context to receive specific recommendations or define next steps based on detected weaknesses. Additionally, it automatically generates versioned technical reports, streamlining the documentation process and improving traceability without requiring manual intervention.

Keywords: Security, Ethical Hacking, Auditing, Automation, Tool.

Índice general

1. Introducción **1**

1.1. Estructura del documento 5

2. Objetivos **7**

2.1. Subobjetivos específicos 8

3. Estado del Arte **10**

3.1. Evolución de las auditorías de seguridad 11

3.2. Herramientas de reconocimiento automatizado 12

3.2.1. ProjectDiscovery Cloud 13

3.2.2. Intruder.io 14

3.2.3. Detectify 16

3.2.4. Propuesta desarrollada 17

3.3. Comparativa de las soluciones analizadas 20

4. Metodología **21**

4.1. Estudio de la fase de reconocimiento 22

4.1.1. Metodologías de referencia 22

4.1.2. Enfoque adoptado en este trabajo 23

4.2. Análisis y elección de herramientas 25

4.2.1. Criterios de selección 25

4.2.2. Comparativa de herramientas consideradas 26

4.3. Resumen de herramientas utilizadas 44

4.4. Flujo metodológico de ejecución 46
4.4.1. Escaneo de puertos y servicios 46
4.4.2. Enumeración del dominio 46
4.4.3. Reconocimiento web 47
4.4.4. Reconocimiento FTP 49
4.4.5. Reconocimiento SSH 49
4.4.6. Generación del informe 49
4.5. Figura resumen del flujo de ejecución 49

5. Implementación y desarrollo **51**
5.1. Diseño de la arquitectura de la herramienta 52
5.1.1. Tecnologías empleadas 57
5.1.2. Comunicación entre componentes 59
5.2. Decisiones técnicas 60
5.2.1. Frontend: HTML, CSS y JavaScript 60
5.2.2. Backend: Python y Flask 61
5.2.3. Orquestación mediante `main.py` 62
5.3. Diseño experimental 62
5.3.1. Requisitos previos 62
5.3.2. Configuración del escaneo 64
5.3.3. Flujo de uso de la herramienta 71

6. Resultados **76**
6.1. Empresa A 77
6.1.1. Subdominio A1 79
6.1.2. Subdominio A2 82
6.1.3. Acciones recomendadas para la Empresa A 85
6.2. Empresa B 86
6.2.1. Acciones recomendadas para la Empresa B 90
6.3. Empresa C 92

6.4. Impacto en la eficiencia del análisis 93
6.4.1. Comparativa por fases 93
6.4.2. Resumen y análisis 98

7. Conclusiones y trabajos futuros **100**
7.1. Conclusiones 101
7.2. Líneas de trabajo futuras 102

Bibliografía **104**

Índice de tablas

3.1. Comparativa entre la herramienta desarrollada y soluciones similares. . . . 20

4.1. Origen metodológico de las tareas incluidas en la herramienta. 24
4.2. Comparativa de herramientas para escaneo de puertos y servicios. 27
4.3. Comparativa de herramientas para reconocimiento de registros DNS. . . . 29
4.4. Comparativa de herramientas para consultas WHOIS. 31
4.5. Comparativa de herramientas para verificación de correos filtrados. 32
4.6. Comparativa de herramientas para enumeración de subdominios. 33
4.7. Comparativa de herramientas para escaneo de vulnerabilidades web. 34
4.8. Comparativa de herramientas para la detección de CMS. 35
4.9. Comparativa de herramientas para la captura automatizada de sitios web. 36
4.10. Comparativa de herramientas para enumeración de directorios web. 38
4.11. Comparativa de herramientas para fuerza bruta SSH. 41
4.12. Comparativa de herramientas para generación de informes PDF. 42

6.1. Resumen del escaneo realizado a la Empresa A. 78
6.2. Resumen del escaneo realizado al subdominio A1. 80
6.3. Resumen del escaneo realizado al subdominio A2. 82
6.4. Resumen del escaneo realizado a la Empresa B. 87

Índice de figuras

1.1. Fases de una auditoría de seguridad ofensiva. 5

4.1. Figura resumen de las herramientas utilizadas en la propuesta. 45
4.2. Flujo metodológico de ejecución de la herramienta. 50

5.1. Tecnologías empleadas en la herramienta 58
5.2. Flujo de comunicación entre los componentes de la herramienta. 60
5.3. Escaneo desde la interfaz gráfica. 65
5.4. Escaneo desde consola. 65
5.5. Escaneo exhaustivo desde la interfaz gráfica. 66
5.6. Formulario de creación de nuevo proyecto. 67
5.7. Consola de escaneo dentro de un proyecto. 68
5.8. Historial de informes generados en un proyecto. 69
5.9. Interacción con el asistente de IA desde un proyecto. 69
5.10. Configuración del proxy desde la interfaz. 70
5.11. Visualización del informe PDF generado. 72
5.12. Generación de siguientes pasos desde una perspectiva ofensiva. 73
5.13. Generación de recomendaciones desde una perspectiva defensiva. 74
5.14. Interacción directa con la IA integrada en la herramienta. 75

6.1. Duración del escaneo realizado a la Empresa A. 79
6.2. Duración del escaneo realizado al subdominio A1. 80
6.3. Puertos abiertos del subdominio A1. 81
6.4. Debilidad de Django encontrada en el subdominio A1. 81

6.5. Duración del escaneo realizado al subdominio A2. 83
6.6. Puertos abiertos del subdominio A2. 83
6.7. Panel de Adminer encontrado en el subdominio A2. 84
6.8. Sugerencias defensivas propuestas por la IA para el subdominio A1. 86
6.9. Duración del escaneo realizado a la Empresa B. 87
6.10. Puertos abiertos de la Empresa B. 88
6.11. Vulnerabilidad SQL Injection de Joomla detectada. 88
6.12. Panel de acceso de Joomla detectado en la Empresa B. 89
6.13. Sugerencias ofensivas propuestas por la IA para la Empresa B. 90
6.14. Sugerencias defensivas propuestas por la IA para la Empresa B. 92
6.15. Tiempo empleado en el escaneo inicial. 94
6.16. Tiempo empleado en el escaneo detallado. 94
6.17. Tiempo de ejecución de Subfinder. 95
6.18. Tiempo de ejecución de Nuclei. 96
6.19. Tiempo de ejecución de Feroxbuster. 97
6.20. Tiempo de ejecución de WhatWeb. 97
6.21. Tiempo de ejecución de JoomScan. 98

Capítulo 1

Introducción

Contenidos

1.1. Estructura del documento 5

Hoy en día, las auditorías de seguridad forman parte esencial de la estrategia de ciberseguridad de cualquier organización. Su finalidad es identificar, evaluar y mitigar vulnerabilidades que puedan ser explotadas por agentes maliciosos. Para lograrlo, estas auditorías se estructuran en distintas fases que emulan el comportamiento de un atacante real, lo que permite analizar el estado de seguridad de los sistemas desde una perspectiva ofensiva.

Una auditoría de seguridad suele desarrollarse de forma secuencial, siguiendo un conjunto de etapas reconocidas en las metodologías profesionales más adoptadas en el sector. Estas fases [1] son las siguientes:

1. **Reconocimiento**: Recopilación pasiva o activa de información sobre el objetivo (direcciones IP, dominios, tecnologías empleadas, subdominios, etc.), que servirá para planificar los ataques posteriores.

2. **Escaneo**: Identificación de servicios activos, versiones, puertos abiertos y posibles vulnerabilidades que puedan ser explotadas.

3. **Obtención de acceso**: Explotación de las vulnerabilidades detectadas en la fase anterior para obtener acceso al sistema, ya sea mediante credenciales débiles, fallos de configuración o ejecución de exploits.

4. **Mantenimiento del acceso**: Aplicación de técnicas para mantener el control del sistema comprometido, como la instalación de puertas traseras (backdoors) o la creación de cuentas persistentes.

5. **Borrado de huellas**: Eliminación de registros, logs y cualquier otra evidencia que pueda alertar a los administradores del sistema de la intrusión realizada.

6. **Informe**: Elaboración de un documento técnico que recoja todo el proceso llevado a cabo, incluyendo las vulnerabilidades encontradas, su criticidad, las evidencias obtenidas y las recomendaciones para corregir los problemas detectados.

Como se puede observar, la fase de **reconocimiento** representa el punto de partida de todo el proceso. A partir de la información recolectada en esta etapa, el auditor podrá construir una visión detallada del entorno y prepararse para fases posteriores como el escaneo o la explotación. Por ello, optimizar esta fase resulta fundamental para garantizar la eficacia y la calidad de toda la auditoría.

Esta fase es una de las más importantes dentro de una auditoría. Tiene como objetivo recopilar la máxima cantidad de información posible sobre el objetivo a analizar [2]. A partir de estos datos, el auditor puede identificar servicios expuestos, tecnologías empleadas y potenciales vectores de ataque, sentando así las bases para etapas más avanzadas como la explotación.

A pesar de su importancia, las tareas asociadas al reconocimiento suelen ser repetitivas, tediosas y propensas a errores si se realizan manualmente. Además, muchas de las herramientas empleadas requieren conocimientos técnicos, uso de línea de comandos y un esfuerzo considerable para gestionar los resultados obtenidos. Este escenario genera una necesidad clara de soluciones que permitan automatizar este proceso, reducir los tiempos de ejecución y facilitar la gestión de la información recopilada.

El presente Trabajo de Fin de Grado surge como respuesta a esta problemática. Se ha desarrollado una herramienta que automatiza las tareas fundamentales de reconocimiento en una auditoría de seguridad, integrando distintas utilidades en un único entorno con interfaz gráfica. Esta solución permite realizar desde escaneos de puertos y servicios hasta la enumeración de subdominios, identificación de vulnerabilidades web, detección de CMS y análisis de servicios como FTP o SSH. Además, la herramienta permite interactuar con un sistema de inteligencia artificial basado en modelos generativos, que facilita la interpretación de resultados y la propuesta de acciones posteriores.

Entre las funcionalidades añadidas destacan también la generación automática y versionada de informes, la configuración de escaneos exhaustivos, la personalización del entorno mediante proxies y un sistema de proyectos que organiza los análisis por objetivo.

La herramienta está diseñada con una arquitectura modular y escalable, lo que permite incorporar en un futuro nuevas funcionalidades de forma sencilla. Esto la convierte en una solución con capacidad para seguir evolucionando y ampliándose mediante la integración de nuevos módulos que permitan cubrir un mayor número de servicios y casos de uso.

Durante el desarrollo se han evaluado múltiples alternativas tecnológicas, priorizando aquellas que ofrecieran un equilibrio entre rendimiento, actualización activa, facilidad de integración y compatibilidad. Herramientas como Nmap, Subfinder, Nuclei, WPScan o Feroxbuster han sido seleccionadas tras un análisis comparativo, mientras que otras como Nikto, Gobuster o Paramiko fueron descartadas por distintos motivos técnicos.

El resultado es una herramienta funcional, útil y extensible, capaz de reducir significativamente el tiempo y la carga de trabajo durante la fase de reconocimiento, agilizando así el proceso de auditoría y mejorando la calidad del análisis desde las primeras etapas.

La figura 1.1 ilustra las distintas fases de una auditoría de seguridad ofensiva, destacando la importancia del reconocimiento como punto de partida y fase en la que se centra esta herramienta.

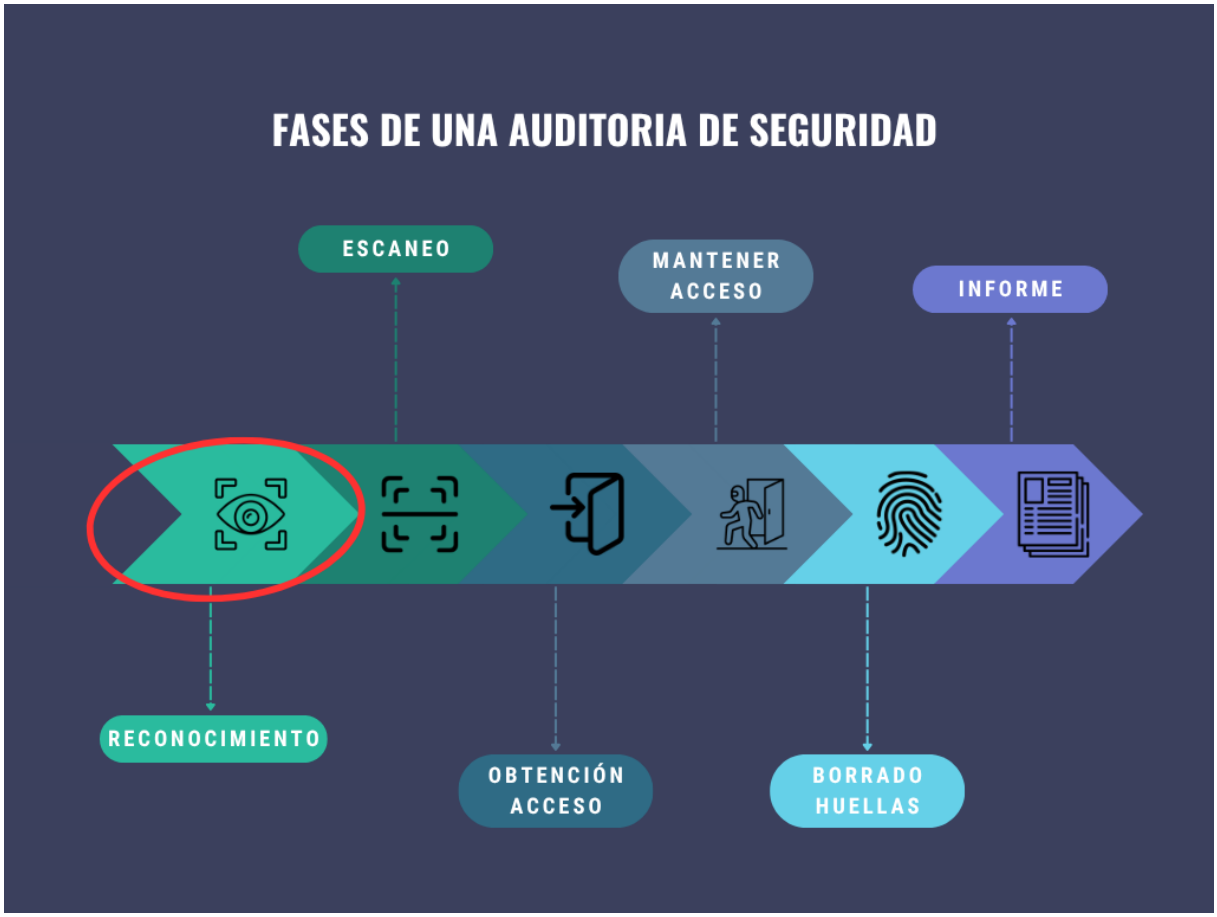


Figura 1.1: Fases de una auditoría de seguridad ofensiva.

1.1. Estructura del documento

Este documento se encuentra estructurado en siete capítulos, además de las correspondientes referencias bibliográficas. A continuación, se describe brevemente el contenido de cada uno de ellos:

- El **Capítulo 2** recoge los objetivos que se pretenden alcanzar con el desarrollo de la herramienta propuesta.
- El **Capítulo 3** presenta el estado del arte, en el que se analizan las principales herramientas existentes en el ámbito del reconocimiento automatizado y se

identifican sus principales limitaciones.

- El **Capítulo 4** describe la metodología seguida para el desarrollo de la herramienta, justificando la elección de cada tecnología empleada y explicando el flujo de trabajo que se implementa durante el proceso de reconocimiento.
- El **Capítulo 5** detalla el proceso de implementación de la herramienta, desde el diseño de su arquitectura hasta la integración de sus distintos módulos.
- El **Capítulo 6** recoge los resultados obtenidos a partir de las pruebas realizadas sobre distintos escenarios, evaluando la eficacia de la herramienta y el cumplimiento de los objetivos definidos.
- El **Capítulo 7** expone las conclusiones extraídas del trabajo, así como posibles líneas futuras de mejora y ampliación del sistema desarrollado.
- Finalmente, se incluyen las **referencias bibliográficas** consultadas a lo largo del proyecto.

Capítulo 2

Objetivos

Contenidos

2.1. Subobjetivos específicos **8**

El principal objetivo de este Trabajo de Fin de Grado consiste en desarrollar una herramienta que automatice las tareas asociadas a la fase de reconocimiento de una auditoría, facilitando y agilizando la recopilación de información sobre un objetivo. Además, esta herramienta deberá ser capaz de generar informes automáticamente a partir de los resultados obtenidos, así como incorporar un asistente de inteligencia artificial para proporcionar recomendaciones contextualizadas que asistan al usuario durante el proceso de auditoría.

2.1. Subobjetivos específicos

Como consecuencia de este objetivo principal, se identifican los siguientes subobjetivos específicos, organizados según las tres funcionalidades clave de la herramienta:

Desarrollo de la herramienta

- Estudiar la metodología seguida en la fase de reconocimiento de una auditoría de seguridad.
- Analizar las distintas alternativas tecnológicas disponibles para cada tarea de reconocimiento, con el fin de seleccionar las opciones más adecuadas en términos de rendimiento, flexibilidad e integración.
- Desarrollar módulos individuales responsables de cada una de las tareas clave que componen la fase de reconocimiento.
- Orquestar las herramientas seleccionadas bajo una única solución que gestione y controle el flujo de ejecución entre los diferentes módulos.
- Permitir la ejecución de la herramienta mediante consola, ampliando su adaptabilidad a distintos perfiles de usuario.

- Diseñar una arquitectura modular y escalable que permita futuras ampliaciones y facilite el mantenimiento del sistema.
- Implementar una interfaz gráfica amigable que simplifique el uso de la herramienta, incluso para usuarios con conocimientos técnicos limitados.
- Incorporar un sistema de gestión de objetivos basado en proyectos, con el objetivo de mejorar la organización y el seguimiento de los análisis realizados.
- Ofrecer distintas opciones de configuración para que la herramienta pueda ajustarse a distintos escenarios de análisis y niveles de profundidad.

Generación automática de informes

- Desarrollar un sistema capaz de generar informes técnicos de forma automática a partir de los resultados obtenidos, presentando la información de manera clara y visual.
- Incluir un sistema de versiones que permita mantener un histórico de todos los informes generados por objetivo.

Integración de inteligencia artificial

- Explorar las características y capacidades de diferentes modelos de inteligencia artificial con el objetivo de identificar aquel que mejor se adapte a las necesidades de este proyecto.
- Incorporar un asistente de inteligencia artificial que proporcione asistencia contextual al usuario y sugerencias sobre acciones posteriores a los resultados obtenidos.

Capítulo 3

Estado del Arte

Contenidos

3.1. Evolución de las auditorías de seguridad 11

3.2. Herramientas de reconocimiento automatizado 12

3.2.1. ProjectDiscovery Cloud 13

3.2.2. Intruder.io 14

3.2.3. Detectify 16

3.2.4. Propuesta desarrollada 17

3.3. Comparativa de las soluciones analizadas 20

3.1. Evolución de las auditorías de seguridad

Las auditorías de seguridad informática surgieron como una respuesta directa al crecimiento exponencial de las redes informáticas y a la necesidad de proteger la información en entornos corporativos y gubernamentales. A medida que las tecnologías digitales comenzaron a formar parte de infraestructuras críticas y operaciones empresariales a gran escala, también aumentaron los incidentes relacionados con accesos no autorizados, robo de información, interrupciones de servicio y ataques maliciosos.

El concepto de auditoría de seguridad se consolidó a finales de los años 70 y principios de los 80, coincidiendo con la adopción masiva de sistemas informáticos interconectados. Durante este periodo, se desarrollaron algunos de los primeros estudios sistemáticos sobre cómo evaluar la seguridad de sistemas informáticos ante accesos no autorizados o comportamientos anómalos. Un ejemplo temprano es el informe *Computer Security Technology Planning Study* elaborado por James P. Anderson en 1972 [3, 4], considerado uno de los documentos fundacionales en la definición de objetivos y estrategias para el análisis técnico de la seguridad.

Durante los años 90, con la expansión global de Internet, surgieron las primeras metodologías estructuradas de auditoría como respuesta a nuevas amenazas, especialmente enfocadas en proteger servidores web, sistemas de correo y bases de datos. Herramientas como `SATAN` (Security Administrator Tool for Analyzing Networks), lanzada en 1995, marcaron un hito como una de las primeras aplicaciones orientadas a la detección de vulnerabilidades de red de forma semiautomática [5].

En paralelo, comenzaron a popularizarse los conceptos de "penetration testing" (pruebas de penetración) y "ethical hacking", que planteaban escenarios controlados de ataque con el fin de identificar debilidades antes de que pudieran ser explotadas por actores maliciosos. Con el tiempo, este enfoque evolucionó en metodologías más completas, como OSSTMM (Open Source Security Testing Methodology Manual) [6] o OWASP Testing Guide [7], que establecieron marcos de referencia para evaluar sistemas desde

una perspectiva técnica y organizada.

Actualmente, las auditorías de seguridad constituyen una práctica esencial dentro de la ciberseguridad defensiva y ofensiva. Su objetivo no se limita únicamente a identificar vulnerabilidades, sino también a priorizarlas, documentarlas y proponer soluciones antes de que puedan ser explotadas en un entorno real. La creciente complejidad de los entornos IT y la aparición de nuevos vectores de ataque han motivado la evolución constante de estas auditorías hacia modelos más automatizados, escalables y adaptables a distintos perfiles de organización.

3.2. Herramientas de reconocimiento automatizado

Con la creciente demanda de eficiencia en los procesos de auditoría de seguridad, especialmente en sus fases iniciales, han surgido en los últimos años múltiples plataformas orientadas a la automatización del reconocimiento y la detección de vulnerabilidades. Estas herramientas buscan reducir la carga operativa del analista, minimizar errores humanos y mejorar la cobertura frente a configuraciones y tecnologías cada vez más diversas.

Una de las propuestas más destacadas en este ámbito es ProjectDiscovery Cloud [8], una plataforma construida sobre herramientas open source ampliamente utilizadas en la comunidad de ciberseguridad, como `Nuclei`, `Httpx`, `Subfinder` o `Dnsx`. Su enfoque se basa en ofrecer un entorno colaborativo, escalable y continuamente actualizado para la detección de vulnerabilidades en activos expuestos a Internet. Aunque su núcleo es técnico, la plataforma proporciona una interfaz web amigable para la gestión de proyectos, permitiendo a los equipos de seguridad integrar fácilmente estos escaneos en flujos de trabajo automatizados.

Otra alternativa relevante es Intruder.io [9], un servicio orientado a empresas que buscan externalizar parte de su vigilancia de ciberseguridad. Intruder.io se centra en

la gestión de vulnerabilidades y la supervisión de la superficie de ataque, integrándose con entornos cloud como AWS o Azure. Su propuesta incluye escaneos automáticos, priorización de riesgos y reportes con recomendaciones técnicas, sin requerir una intervención manual intensiva, aunque con una menor flexibilidad técnica en comparación con herramientas de línea de comandos.

Por su parte, Detectify [10] ofrece un enfoque complementario basado en la inteligencia colectiva, donde una comunidad de hackers éticos colabora aportando módulos de detección de vulnerabilidades, incluyendo incluso algunos 0-days [11]. Esta plataforma permite escaneos web completos, incluyendo aplicaciones protegidas por autenticación, y destaca por su integración con herramientas de desarrollo y gestión como GitHub, Jira o Slack. Además, incorpora técnicas de fingerprinting avanzadas para adaptar sus pruebas al stack tecnológico específico de cada objetivo.

3.2.1. ProjectDiscovery Cloud

ProjectDiscovery Cloud es una plataforma de seguridad ofensiva que se construye sobre herramientas de código abierto ampliamente adoptadas por la comunidad de pentesting, como `Nuclei`, `Httpx`, `Subfinder`, `Naabu` o `Dnsx`. Su objetivo principal es facilitar la automatización del reconocimiento y la identificación de vulnerabilidades en activos expuestos, ofreciendo una experiencia escalable, colaborativa y fácil de integrar en flujos de trabajo modernos.

Entre sus principales características destacan:

- **Escaneo de puertos y servicios**: utiliza `Naabu` para detectar puertos abiertos en los objetivos definidos, permitiendo descubrir servicios accesibles desde Internet.
- **Enumeración de subdominios**: se apoya en `Subfinder` para identificar subdominios del dominio objetivo, lo cual permite ampliar la superficie de ataque.
- **Escaneo de vulnerabilidades web**: emplea `Nuclei` como motor principal para

ejecutar pruebas basadas en plantillas YAML contra los activos detectados. Estas plantillas permiten buscar fallos de seguridad en servicios HTTP, encabezados, tecnologías, versiones o patrones de exposición comunes.

- **Detección de tecnologías**: con herramientas como `Httpx`, la plataforma identifica frameworks, CMS o servidores web en uso.
- **Gestión de proyectos**: permite organizar los objetivos dentro de espacios de trabajo, visualizar resultados agrupados y priorizar vulnerabilidades según su criticidad.
- **Interfaz web y CLI**: proporciona tanto una interfaz gráfica intuitiva como una línea de comandos para ejecutar las herramientas por separado o integrarlas en scripts.

Sin embargo, la plataforma tiene ciertas limitaciones. Por ejemplo, no incluye reconocimiento detallado de servicios específicos como FTP o SSH, ni ofrece funciones de OSINT orientadas a la detección de correos filtrados o información expuesta de usuarios. Tampoco integra asistentes inteligentes para analizar los resultados, ni proporciona informes de forma nativa.

En cuanto a la disponibilidad, ProjectDiscovery Cloud ofrece una modalidad gratuita con funcionalidades limitadas, ideal para usuarios técnicos que ya están familiarizados con sus herramientas open source. Los planes comerciales amplían capacidades como la colaboración multiusuario o el acceso a plantillas explusivas.

3.2.2. Intruder.io

Intruder.io es una plataforma de escaneo de vulnerabilidades enfocada en entornos empresariales, cuyo objetivo principal es simplificar la supervisión de la superficie de ataque y automatizar la detección de debilidades técnicas. Su propuesta se centra

en ofrecer una experiencia gestionada, accesible a equipos de TI y seguridad que no necesariamente cuentan con perfiles técnicos especializados.

Las características principales de Intruder.io incluyen:

- **Escaneo de puertos y servicios**: permite identificar puertos abiertos y servicios disponibles en los activos definidos, tanto en entornos externos como internos. Esta capacidad es parte de su sistema de supervisión continua.
- **Enumeración de subdominios**: incluye detección automática de subdominios asociados a dominios registrados, para ampliar la visibilidad sobre activos inadvertidos.
- **Escaneo de vulnerabilidades web**: utiliza una base de pruebas automatizadas para detectar vulnerabilidades comunes en aplicaciones web, como inyecciones, XSS o configuraciones inseguras.
- **Gestión de activos y proyectos**: organiza los resultados por organización y objetivos, permitiendo la priorización basada en el riesgo, e integración con herramientas como AWS o Azure para escanear nuevos servicios desplegados automáticamente.
- **Interfaz web**: proporciona una interfaz gráfica sencilla, centrada en accesibilidad y claridad de resultados.

A pesar de las características previamente mencionadas, Intruder.io presenta varias limitaciones comparado con la solución propuesta en este trabajo. No permite ejecución mediante línea de comandos, lo cual restringe su integración en flujos automatizados personalizados. Tampoco realiza reconocimiento específico sobre servicios como FTP o SSH, ni incluye análisis de correos expuestos o técnicas de OSINT. Además, no ofrece análisis mediante inteligencia artificial para la interpretación de los resultados obtenidos.

Su modelo de uso se basa principalmente en suscripción comercial. No dispone de una versión gratuita para auditorías técnicas avanzadas, lo que puede suponer una barrera de entrada para usuarios individuales.

3.2.3. Detectify

Detectify es una plataforma de escaneo de seguridad web orientada a organizaciones que necesitan identificar y corregir vulnerabilidades en aplicaciones expuestas a Internet. Su enfoque combina el uso de pruebas automatizadas con inteligencia de amenazas aportada por una comunidad de más de 400 hackers éticos, quienes contribuyen con módulos de detección actualizados, incluyendo vulnerabilidades recientes e incluso 0-days.

Entre sus principales funcionalidades destacan:

- **Escaneo de puertos y servicios**: realiza detección básica de servicios expuestos, aunque centrado principalmente en protocolos web.
- **Enumeración de subdominios**: detecta subdominios relacionados con el dominio objetivo, y los incluye como parte del análisis continuo de la superficie de ataque.
- **Reconocimiento DNS**: ofrece visibilidad sobre registros DNS, lo que permite detectar configuraciones erróneas y posibles riesgos de seguridad.
- **Detección de tecnologías y CMS**: realiza fingerprinting de tecnologías, frameworks y gestores de contenido como WordPress, Joomla o Drupal, adaptando las pruebas de seguridad a cada stack detectado.
- **Escaneo de vulnerabilidades web**: emplea pruebas dinámicas (DAST) para identificar fallos como XSS, inyecciones SQL, exposición de archivos sensibles y configuraciones inseguras.
- **IA para análisis y priorización**: recientemente ha incorporado una herramienta basada en inteligencia artificial, *Alfred*, que ayuda a priorizar vulnerabilidades y

generar pruebas automatizadas.

- **Gestión de activos y generación de informes**: permite agrupar objetivos, gestionar su historial de escaneos y generar informes detallados con recomendaciones técnicas. Estos informes son exportables y compatibles con herramientas como Jira, Slack o GitHub.

Detectify ofrece una experiencia potente y pulida en el ámbito web, especialmente adecuada para equipos de desarrollo que integran la seguridad dentro del ciclo de vida del software (DevSecOps). No obstante, presenta ciertas limitaciones cuando se compara con herramientas orientadas a auditorías ofensivas más amplias. No dispone de ejecución mediante línea de comandos, ni permite escanear servicios que no sean web, como FTP o SSH.

Otro aspecto relevante es que Detectify no ofrece una capa gratuita, lo que lo posiciona claramente como una solución orientada a clientes corporativos con necesidades específicas y presupuesto disponible.

3.2.4. Propuesta desarrollada

La herramienta desarrollada en este Trabajo de Fin de Grado nace con el objetivo de cubrir de forma integral y automatizada la fase de reconocimiento dentro de una auditoría de seguridad. Para ello, se ha diseñado un sistema modular, flexible y extensible que agrupa múltiples funcionalidades bajo una única interfaz, permitiendo al auditor realizar análisis técnicos avanzados sin necesidad de recurrir a múltiples herramientas independientes.

A continuación se detallan sus principales capacidades:

- **Escaneo de puertos y servicios**: utiliza `Nmap` para detectar puertos abiertos, servicios en ejecución y versiones asociadas.

- **Reconocimiento DNS**: integra herramientas como `DNSdumpster`, `MXToolbox` y `API Ninja (WHOIS)` para obtener información completa sobre registros DNS, políticas de correo (`DMARC`), y datos administrativos del dominio.
- **OSINT sobre correos electrónicos**: combina `IntelligenceX` y `LeakCheck` para localizar posibles direcciones de correo filtradas y verificar su aparición en brechas de seguridad públicas.
- **Enumeración de subdominios**: mediante `Subfinder` y `HTTPX`, identifica subdominios expuestos y verifica su disponibilidad mediante solicitudes HTTP.
- **Escaneo de vulnerabilidades web**: emplea `Nuclei`, tanto sobre el dominio principal como sobre los subdominios (si se activa el modo exhaustivo).
- **Detección de CMS**: con `WhatWeb`, identifica si el sitio web utiliza un gestor de contenidos (como WordPress o Joomla), para adaptar las siguientes fases del análisis.
- **Reconocimiento específico por CMS**:
  - **WordPress**: utiliza `WPScan` para detectar usuarios, plugins instalados y vulnerabilidades conocidas. También lanza ataques de fuerza bruta sobre `xmlrpc.php`.
  - **Joomla**: ejecuta `Joomscan` para identificar vulnerabilidades específicas del CMS, archivos expuestos y configuraciones inseguras.
- **Enumeración de directorios y archivos**: lanza `Feroxbuster` para realizar *fuzzing* de rutas web. Si se activa el modo exhaustivo, incrementa la recursividad para profundizar el análisis.
- **Reconocimiento de servicios específicos**:
  - **FTP**: analiza si permite acceso anónimo, realiza fuerza bruta con `ftplib`, verifica permisos y descarga el contenido disponible.
  - **SSH**: realiza ataques de fuerza bruta con `Hydra` y utiliza `Searchsploit` para identificar vulnerabilidades según la versión detectada.

- **Captura de pantalla del sitio web**: mediante `Pyppeteer`, captura una imagen del sitio web objetivo, útil para documentar su estado en los informes.
- **Generación de informes**: crea documentos PDF automáticos con los resultados del análisis, integrando capturas, tablas y hallazgos organizados por fases.
- **Gestión de proyectos**: permite crear, guardar y versionar escaneos asociados a un mismo objetivo, lo que facilita el seguimiento de la evolución del sistema a lo largo del tiempo.
- **Interfaz gráfica**: proporciona una interfaz intuitiva desde la cual se pueden gestionar proyectos, lanzar escaneos y consultar resultados, sin necesidad de conocimientos técnicos avanzados.
- **Ejecución por consola**: también puede ejecutarse vía terminal, permitiendo su integración en otros flujos o entornos automatizados. Ofrece parámetros como `-p` para asociar un escaneo a un proyecto y `-ex` para activar el modo exhaustivo.
- **Modo de escaneo exhaustivo**: activa configuraciones más profundas para ciertos módulos, como lanzar `Nuclei` sobre todos los subdominios o aumentar la recursividad de `Feroxbuster`.
- **Configuración de proxy**: desde la interfaz gráfica se puede definir un proxy personalizado que será utilizado por herramientas compatibles como `WPScan`, reduciendo así el riesgo de bloqueos por parte de los servicios analizados.
- **Asistente de IA**: integra un modelo generativo (`Gemini`) que permite al usuario:
  - **Consultar recomendaciones y próximos pasos**: basados en los resultados del escaneo.
  - **Obtener sugerencias de soluciones**: adaptadas a las vulnerabilidades detectadas.

Esta combinación de funcionalidades técnicas, automatización, opciones de ejecución y asistencia mediante inteligencia artificial posiciona a la herramienta como una solución diferencial frente a otras propuestas existentes.

3.3. Comparativa de las soluciones analizadas

La Tabla 3.1 resume las capacidades clave de cada herramienta comparada, destacando tanto las funcionalidades comunes como las características diferenciales que aporta la solución desarrollada.

Característica	Propuesta	ProjectDiscovery Cloud	Intruder.io	Detectify
Escaneo de puertos y servicios	✓	✓	✓	✓
Enumeración de subdominios	✓	✓	✓	✓
Reconocimiento DNS	✓	✗	✗	✓
Escaneo específico por CMS	✓	✗	✗	✓
Escaneo de vulnerabilidades web	✓	✓	✓	✓
Reconocimiento específico de servicios	✓	✗	✗	✗
OSINT sobre direcciones de correo	✓	✗	✗	✗
Gestión de proyectos/objetivos	✓	✓	✓	✓
Generación de reportes versionados	✓	✗	✓	✓
IA integrada para asistencia y análisis	✓	✗	✗	✓
Interfaz web	✓	✓	✓	✓
Ejecución por CLI	✓	✓	✗	✗
Servicio gratuito	✓	✓	✗	✗

Tabla 3.1: Comparativa entre la herramienta desarrollada y soluciones similares.

Tras revisar las principales plataformas orientadas al reconocimiento automatizado en auditorías de seguridad, se observa que, si bien ofrecen soluciones potentes en distintos aspectos, presentan limitaciones significativas en cuanto a personalización, cobertura de vectores no web, o interacción con los resultados obtenidos. Estas carencias evidencian la necesidad de desarrollar herramientas más flexibles y adaptadas a un análisis técnico más profundo, como la que se propone en este trabajo.

Capítulo 4

Metodología

Contenidos

4.1. Estudio de la fase de reconocimiento 22

4.1.1. Metodologías de referencia 22

4.1.2. Enfoque adoptado en este trabajo 23

4.2. Análisis y elección de herramientas 25

4.2.1. Criterios de selección 25

4.2.2. Comparativa de herramientas consideradas 26

4.3. Resumen de herramientas utilizadas 44

4.4. Flujo metodológico de ejecución 46

4.4.1. Escaneo de puertos y servicios 46

4.4.2. Enumeración del dominio 46

4.4.3. Reconocimiento web 47

4.4.4. Reconocimiento FTP 49

4.4.5. Reconocimiento SSH 49

4.4.6. Generación del informe 49

4.5. Figura resumen del flujo de ejecución 49

Tal y como se ha mencionado en capítulos anteriores, la fase de reconocimiento constituye un componente fundamental dentro de cualquier auditoría de seguridad. Su correcta ejecución permite identificar activos expuestos, tecnologías empleadas y posibles vectores de ataque. Por este motivo, resulta esencial contar con una metodología adecuada que permita abordar esta fase de forma eficiente, estructurada y efectiva.

4.1. Estudio de la fase de reconocimiento

La fase de reconocimiento en una auditoría de seguridad consiste en el conjunto de técnicas empleadas para recopilar, de forma activa o pasiva, información relevante sobre un objetivo. Dicha información puede incluir aspectos como servicios expuestos, dominios relacionados, tecnologías utilizadas, direcciones de correo filtradas o configuraciones vulnerables. Esta etapa inicial es crucial, ya que establece las bases para el resto del proceso de auditoría: a partir de estos datos se pueden identificar posibles vectores de ataque, evaluar superficies expuestas y priorizar futuras acciones ofensivas o defensivas.

Al tratarse de una fase con un alto grado de repetitividad y lógica condicional (por ejemplo, "si el puerto 21 está abierto, normalmente asociado al servicio `FTP`, comprobar acceso anónimo"), es especialmente adecuada para ser automatizada. Automatizarla no solo ahorra tiempo y esfuerzo humano, sino que mejora la consistencia y trazabilidad de los resultados obtenidos, al aplicar siempre el mismo flujo metodológico ante situaciones similares.

4.1.1. Metodologías de referencia

Para establecer un marco sólido sobre el cual diseñar el flujo de reconocimiento implementado, se han estudiado tres metodologías ampliamente aceptadas en el ámbito de la ciberseguridad:

- **OWASP Web Security Testing Guide (WSTG)** [12]: guía enfocada a la seguridad de aplicaciones web. Incluye en sus primeras fases una serie de pruebas relacionadas con la recolección de información (como descubrimiento de subdominios, fingerprinting de tecnologías, análisis de DNS o WHOIS). Su orientación a entornos web y enfoque sistemático la convierten en una referencia especialmente útil para este proyecto.

- **MITRE ATTACK Framework** [13]: base de conocimiento que documenta técnicas utilizadas por profesionales en el mundo real. La táctica de *Reconnaissance* incluye técnicas como escaneo activo, recolección de DNS, WHOIS lookup, entre otras. Su grado de detalle la convierten en una fuente ideal para este proyecto.

- **OSSTMM (Open Source Security Testing Methodology Manual)** [14]: metodología de auditoría que organiza el análisis por canales de interacción y define la enumeración como una parte esencial del proceso de reconocimiento. Aunque es más generalista y menos enfocada al entorno web, proporciona una base de conocimientos sólida.

4.1.2. Enfoque adoptado en este trabajo

Las tres metodologías han sido consideradas como referencia durante el diseño de la herramienta, y en cierto modo, se han aplicado facetas de todas ellas. No obstante, el enfoque metodológico seguido no se ha adherido estrictamente a ninguna de ellas de forma literal. En su lugar, se ha optado por una aplicación flexible, combinando lo más relevante de cada marco con criterios prácticos derivados de la experiencia técnica y del objetivo del proyecto.

Concretamente, el diseño del flujo de reconocimiento se apoya principalmente en las directrices del marco OWASP, dada su orientación clara a entornos web, y en la estructura del marco MITRE ATTACK, por su nivel de detalle y organización táctica. La herramienta incorpora fases descritas en ambos marcos, como el fingerprinting de tecnologías, el escaneo

de vulnerabilidades web o la recopilación de información DNS y WHOIS.

Sin embargo, también se han añadido tareas no descritas explícitamente en dichos marcos, como la enumeración de subdominios o el análisis de servicios específicos, por considerarse elementos fundamentales en un reconocimiento técnico completo. Del mismo modo, algunas técnicas presentes en estas metodologías han sido descartadas por no ajustarse al alcance funcional de esta herramienta.

En la Tabla 4.1 se incluye la metodología de origen de cada una de las tareas que incorpora la herramienta. Cabe destacar que aquellas tareas cuyo origen no se corresponde con ninguna de las metodologías de referencia se encuentran marcadas como **UEx**.

Tarea	**Origen metodológico**
Escaneo de puertos y servicios	MITRE
Reconocimiento DNS	MITRE
Email OSINT	MITRE
Enumeración de subdominios	UEx
Escaneo de vulnerabilidades web	OWASP / MITRE
Detección de CMS	OWASP
Captura de pantalla del sitio web	UEx
Enumeración de directorios y archivos web	OWASP / MITRE
Reconocimiento WordPress	OWASP / MITRE
Reconocimiento Joomla	OWASP / MITRE
Reconocimiento FTP	UEx
Reconocimiento SSH	UEx

Tabla 4.1: Origen metodológico de las tareas incluidas en la herramienta.

Las fases implementadas se documentan con mayor detalle en los apartados siguientes,

donde se describen las herramientas y tecnologías utilizadas en cada etapa, así como el flujo metodológico exacto que sigue la herramienta.

4.2. Análisis y elección de herramientas

La elección de herramientas adecuadas resulta clave para garantizar la eficacia, eficiencia y fiabilidad. En el desarrollo de este proyecto, se han analizado diversas alternativas tecnológicas disponibles para cada una de las tareas que conforman la fase de reconocimiento, valorando sus prestaciones, grado de actualización, facilidad de integración, rendimiento y adecuación al flujo metodológico propuesto.

Este apartado presenta, en primer lugar, los criterios que han guiado la selección de dichas herramientas. Posteriormente, se comparan distintas opciones consideradas para cada funcionalidad, justificando la decisión final adoptada en cada caso.

4.2.1. Criterios de selección

Para la elección de la herramienta más adecuada en cada una de las fases del reconocimiento, se han definido una serie de criterios con el fin de asegurar su alineación con los objetivos de la solución desarrollada. Estos criterios se han priorizado en función de su relevancia dentro del contexto del proyecto, siendo los siguientes, ordenados de mayor a menor importancia:

- **Grado de reconocimiento**: Se valora el uso extendido y el reconocimiento de la herramienta en entornos profesionales.
- **Facilidad de integración**: Se seleccionan herramientas que puedan integrarse de manera sencilla en el flujo de ejecución de la solución, preferiblemente aquellas que ofrezcan soporte en línea de comandos, salida en formatos tratables (JSON, CSV) o APIs.

- **Rendimiento**: Se evalúa la eficiencia en el uso de recursos y la velocidad de ejecución.
- **Actualización activa**: Se priorizan herramientas con desarrollo activo y mantenimiento continuo, así como con una comunidad de usuarios que contribuya regularmente a su evolución.
- **Flexibilidad**: Se prefieren herramientas que permitan ajustes de configuración, modo de ejecución, profundidad de análisis y adaptación a distintos escenarios de uso.
- **Documentación**: Se considera importante la existencia de una documentación clara, actualizada y accesible, que facilite tanto su uso como su integración y mantenimiento.

A partir de estos criterios, se ha llevado a cabo un análisis exhaustivo de las distintas alternativas disponibles para cada etapa del reconocimiento. Las decisiones adoptadas, así como las herramientas finalmente seleccionadas, se presentan y justifican en los apartados siguientes.

4.2.2. Comparativa de herramientas consideradas

Una vez definidos los criterios de selección, en este apartado se presenta la toma de decisiones llevada a cabo para la elección de cada una de las herramientas de las distintas fases.

Escaneo de puertos y servicios

Para la etapa de escaneo de puertos y servicios se ha seleccionado **Nmap** [15], una de las herramientas más consolidadas y ampliamente utilizadas en el ámbito de la ciberseguridad. Su elección fue clara debido a su reconocimiento como estándar para el

descubrimiento de puertos abiertos, detección de servicios y recopilación de información sobre los sistemas analizados.

Nmap destaca por su robustez, su comunidad activa de usuarios, la disponibilidad de documentación extensa y su versatilidad para adaptarse a distintos niveles de profundidad en el análisis. Además, permite realizar detección de versiones y ejecución de scripts específicos mediante su motor NSE (Nmap Scripting Engine), funcionalidades que otras alternativas no alcanzan al mismo nivel.

Durante el proceso de selección se valoraron también otras herramientas como **Masscan** [16], reconocida por su velocidad de escaneo en redes de gran tamaño, pero que no ofrece detección de servicios ni análisis detallado. Asimismo, se consideró **RustScan** [17], una herramienta moderna que destaca por su rapidez y eficiencia, pero que no cuenta con la misma consolidación ni comunidad que Nmap.

La Tabla 4.2 resume la comparativa entre estas tres herramientas aplicando los criterios definidos anteriormente.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
Nmap	★★★	★★★	★★☆	★★★	★★★	★★★
Masscan	★★☆	★★☆	★★★	★★☆	★☆☆	★☆☆
RustScan	★★☆	★★★	★★★	★★★	★★☆	★★★

Tabla 4.2: Comparativa de herramientas para escaneo de puertos y servicios.

Teniendo en cuenta esto, **Nmap** ha sido la opción seleccionada para cubrir esta fase, por ser la herramienta más completa, consolidada y adecuada para los objetivos del proyecto.

Reconocimiento DNS

La etapa de reconocimiento DNS tiene como objetivo recopilar la mayor cantidad de información posible relacionada con la infraestructura DNS del objetivo. Para ello se han utilizado tres herramientas, cada una especializada en una funcionalidad específica:

- **DNSdumpster** [18]: empleada para consultar y visualizar registros DNS públicos del dominio, incluyendo registros `MX` (servidores de correo), `NS` (servidores de nombres) y `TXT` (configuraciones SPF, DKIM, etc.).
- **MXToolbox** [19]: utilizada específicamente para la consulta del registro `DMARC`, el cual proporciona mecanismos de protección frente a ataques de suplantación de identidad mediante correo electrónico (*email spoofing* [20]).
- **API Ninja (WHOIS)** [21]: empleada para realizar consultas WHOIS y obtener información sobre la propiedad del dominio, la fecha de creación y expiración, así como otros datos administrativos relevantes.

Durante el proceso de selección de estas herramientas, se consideraron varias alternativas que ofrecían funcionalidades similares.

En primer lugar, se necesitaba una herramienta que permitiera consultar registros DNS públicos. En este caso se optó por **DNSdumpster**, que destaca por su facilidad de uso e integración gracias a la amplia capa gratuita que ofrece su API. Aunque existen otras herramientas como **dnsrecon** o **dnsenum**, se descartaron por las siguientes razones:

- **dnsrecon** [22]: ofrece buenos resultados, pero su integración en la herramienta resultaba menos práctica, ya que no genera salidas en formatos estructurados como JSON.
- **dnsenum** [23]: no proporciona tanta información como DNSdumpster y presenta un mantenimiento limitado, con escasa actividad reciente en su desarrollo.

- **Amass** [24]: una herramienta potente y moderna, pero con una integración algo más compleja.

La Tabla 4.3 resume la comparativa entre estas herramientas siguiendo los criterios definidos anteriormente.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
DNSdumpster	★★☆	★★★	★★★	★★★	★★★	★★★
Dnsrecon	★★☆	★☆☆	★★☆	★★☆	★★☆	★★★
Dnsenum	★★☆	★☆☆	★★☆	★☆☆	★★☆	★★★
Amass	★★☆	★★☆	★★★	★★☆	★★★	★★★

Tabla 4.3: Comparativa de herramientas para reconocimiento de registros DNS.

La facilidad de integración, la amplia capa gratuita de su API y la calidad de la información obtenida llevaron a seleccionar **DNSdumpster** como la herramienta más adecuada para cumplir esta función.

Otra parte fundamental dentro del reconocimiento DNS es la consulta del registro `DMARC`. El mantenimiento y la monitorización de este registro son aspectos clave para garantizar la autenticidad del dominio y protegerlo frente a ataques de suplantación de identidad mediante correo electrónico.

Para esta tarea se optó por utilizar **MXToolbox** debido a su simplicidad de uso y a la disponibilidad de su funcionalidad DMARC a través de una API para realizar la consulta. Aunque existen otras plataformas como **EasyDMARC** [25], estas no ofrecen una API pública, lo que complica su integración en flujos automatizados como el propuesto en esta herramienta.

El número de herramientas con esta funcionalidad específica y con capacidad de integración es muy limitado, lo que llevó a seleccionar **MXToolbox** como la opción más adecuada.

Para finalizar el reconocimiento DNS, se requería de una herramienta capaz de realizar consultas WHOIS. Esta funcionalidad es esencial para obtener información sobre la propiedad del dominio, fechas de creación y expiración, así como otros datos que pueden resultar útiles en fases posteriores de análisis.

Durante el proceso de evaluación se analizaron diferentes alternativas, entre las cuales destacan las siguientes:

- **Comando `whois`**: herramienta disponible desde línea de comandos. Fue descartada debido a que su salida no es fácilmente tratable (por ejemplo, no permite exportar directamente en formatos como JSON o CSV), lo que dificulta su integración en flujos de trabajo automatizados.
- **Who.is** [26]: plataforma web que permite realizar consultas manuales a través de su interfaz gráfica. No ofrece una API pública, por lo que resulta inviable para una solución automatizada.
- **WhoisXML API** [27]: solución con una API REST bien estructurada y documentada. No obstante, su capa gratuita es bastante limitada en cuanto a número de consultas, lo que redujo su viabilidad para el contexto del proyecto.

Finalmente, se optó por la funcionalidad WHOIS de **API Ninja** [21], que ofrece una API REST con una capa gratuita muy completa, respuestas en formato JSON y una integración sencilla. Estas características la convierten en una solución ideal para la herramienta desarrollada.

La Tabla 4.4 resume la comparativa entre las principales opciones evaluadas.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
API Ninja (WHOIS)	★★☆	★★★	★★★	★★★	★★★	★★★
CLI `whois`	★★★	★☆☆	★★★	★★☆	★☆☆	★☆☆
Who.is	★★☆	★☆☆	★★★	★★★	★☆☆	★★☆
WhoisXML API	★★☆	★★★	★★★	★★★	★★☆	★★★

Tabla 4.4: Comparativa de herramientas para consultas WHOIS.

OSINT de correos electrónicos

La herramienta incorpora una fase de OSINT centrada en el análisis de direcciones de correo electrónico asociadas al dominio objetivo. Este análisis permite identificar exposiciones de correos en bases de datos filtradas, lo que puede revelar vectores de ataque adicionales como credenciales comprometidas, datos personales o relaciones con otros servicios.

Para llevar a cabo esta tarea, se ha optado por combinar dos herramientas:

- **IntelligenceX** [28]: encargada de obtener direcciones de correo electrónico relacionadas con el dominio analizado a partir de bases de datos públicas y documentos expuestos. Su funcionalidad resulta clave para automatizar la recopilación inicial de correos que podrían haber sido filtrados.
- **LeakCheck** [29]: utilizada para comprobar si las direcciones encontradas han sido comprometidas en alguna brecha de seguridad conocida. La herramienta devuelve información detallada sobre la filtración y los datos expuestos.

En el caso de **IntelligenceX**, no se identificaron alternativas que ofrecieran una funcionalidad equivalente de forma gratuita y con una API pública abierta, por lo que se optó por ésta.

Para la verificación de brechas se evaluaron diferentes opciones. Las principales

alternativas a **LeakCheck** fueron:

- **Have I Been Pwned (HIBP)** [30]: ampliamente reconocida como referencia en este ámbito, pero su API requiere una suscripción de pago, lo que dificulta su integración en la herramienta.
- **BreachDirectory y similares** [31]: plataformas que ofrecen servicios similares a HIBP, pero con APIs gratuitas, aunque con un número de consultas muy limitado.

Finalmente, se optó por **LeakCheck**, ya que ofrece una API gratuita con un número de consultas adecuado para el contexto del proyecto. Además, su integración es sencilla y proporciona información sobre las brechas de seguridad en las que se han visto comprometidos los correos electrónicos analizados.

La Tabla 4.5 presenta una comparativa de estas tres soluciones según los criterios definidos para el proyecto.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
LeakCheck	★★☆	★★★	★★★	★★★	★★★	★★★
Have I Been Pwned	★★★	★★☆	★★★	★★★	★★☆	★★★
BreachDirectory y similares	★★☆	★★☆	★★★	★★★	★★☆	★★★

Tabla 4.5: Comparativa de herramientas para verificación de correos filtrados.

Enumeración de subdominios

Esta etapa tiene como objetivo identificar subdominios asociados al dominio principal del objetivo. Esta fase permite ampliar la superficie de ataque y descubrir servicios que pueden no estar expuestos de forma directa o evidente.

Para llevar a cabo esta tarea, se ha optado por combinar dos herramientas:

- **Subfinder** [32]: herramienta de código abierto desarrollada por ProjectDiscovery,

que permite enumerar subdominios a partir de múltiples fuentes públicas, como motores de búsqueda, certificados SSL, DNS y archivos históricos.

- **HTTPX** [33]: utilizada para verificar la validez de los subdominios identificados por Subfinder. Esta herramienta permite comprobar si los subdominios están activos mediante peticiones HTTP, validando así su accesibilidad real.

Durante el proceso de evaluación se consideraron otras alternativas, como **crt.sh** [34], un buscador de certificados SSL que permite identificar subdominios registrados en certificados públicos. No obstante, esta opción fue descartada debido a la falta de una API pública, lo que dificultaba su integración en flujos de análisis automatizados.

La Tabla 4.6 presenta una comparativa entre las principales opciones evaluadas para la enumeración de subdominios.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
Subfinder	★★★	★★★	★★★	★★★	★★★	★★★
crt.sh	★★☆	★☆☆	★★☆	★★☆	★☆☆	★★☆

Tabla 4.6: Comparativa de herramientas para enumeración de subdominios.

El alto reconocimiento por parte de la comunidad *Red Team* [35], junto con su versatilidad y facilidad de integración, llevaron a seleccionar **Subfinder** como la herramienta principal para la enumeración de subdominios.

Por su parte, se eligió **HTTPX** para verificar la validez de los subdominios identificados, al formar parte de la misma suite de herramientas de ProjectDiscovery, contar con una biblioteca en Python fácilmente integrable y ofrecer un rendimiento elevado en tareas de validación HTTP.

Escaneo de vulnerabilidades web

La etapa de escaneo de vulnerabilidades web tiene como objetivo detectar debilidades presentes en los servicios web identificados. Para esta fase se ha seleccionado **Nuclei** [36], una herramienta de código abierto desarrollada por ProjectDiscovery que permite identificar vulnerabilidades en aplicaciones web mediante el uso de plantillas altamente personalizables. Estas plantillas, escritas en YAML, cubren desde pruebas de seguridad básicas hasta la detección de configuraciones erróneas, vulnerabilidades específicas de versiones, fallos en encabezados HTTP, entre otros.

Durante el proceso de selección se valoraron algunas alternativas, entre las cuales se analizó **Nikto** [37], una herramienta clásica para análisis de servidores web. Sin embargo, esta opción fue descartada al no ofrecer el mismo nivel de modularidad, flexibilidad ni personalización que Nuclei. Además, su base de pruebas está orientada a configuraciones obsoletas y su desarrollo es menos activo.

La Tabla 4.7 muestra la comparativa entre ambas herramientas según los criterios definidos.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
Nuclei	★★★	★★★	★★★	★★★	★★★	★★★
Nikto	★★☆	★★☆	★★☆	★★☆	★☆☆	★★★

Tabla 4.7: Comparativa de herramientas para escaneo de vulnerabilidades web.

No se estudiaron más opciones ya que desde el inicio se consideró prioritaria la integración de **Nuclei** debido a su auge actual, su rendimiento y su gran comunidad de usuarios en constante crecimiento.

Detección de CMS

Para la detección del sistema de gestión de contenidos (CMS) utilizado por el sitio web se ha seleccionado **WhatWeb** [38], una herramienta ampliamente reconocida que permite identificar tecnologías utilizadas en una página web, incluyendo el CMS y otros frameworks comunes. Esta fase resulta clave, ya que en función del CMS detectado se activan módulos específicos orientados a su análisis y explotación.

Al igual que en la fase de escaneo de vulnerabilidades web, desde el inicio del desarrollo se consideró prioritaria la elección de **WhatWeb** debido a su amplio reconocimiento en la comunidad de ciberseguridad, su rendimiento y su facilidad de integración mediante línea de comandos.

No obstante, se evaluaron también otras alternativas, entre ellas:

- **CMSeeK** [39]: herramienta especializada en detección de CMS. A pesar de su enfoque, fue descartada por su menor flexibilidad, su limitada documentación y su falta de mantenimiento reciente.
- **Wappalyzer (CLI)** [40]: versión en línea de comandos del conocido motor utilizado en su extensión de navegador. Si bien la versión web cuenta con un alto grado de precisión y popularidad, su implementación en CLI presenta menor soporte, menos documentación y una comunidad de usuarios significativamente más reducida.

La Tabla 4.8 resume la comparativa entre estas herramientas según los criterios definidos en el proyecto.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
WhatWeb	★★★	★★★	★★★	★☆☆	★★★	★★★
CMSeeK	★☆☆	★★☆	★★☆	★☆☆	★★☆	★★☆
Wappalyzer CLI	★★☆	★★☆	★★☆	★★☆	★★☆	★☆☆

Tabla 4.8: Comparativa de herramientas para la detección de CMS.

Por tanto, se optó por **WhatWeb** como herramienta principal para la detección de CMS, debido a su integración sencilla, alto grado de reconocimiento y buen rendimiento en entornos automatizados.

Captura de pantalla del sitio web

La herramienta incorpora una fase en la que se realiza una captura de pantalla del sitio web analizado, con el objetivo de documentar visualmente su apariencia y adjuntarla en el informe generado. Esta funcionalidad es especialmente útil, ya que permite conservar evidencia visual del estado del sitio en el momento del análisis.

Para llevar a cabo esta tarea se ha seleccionado **Pyppeteer** [41], una biblioteca de Python que permite controlar navegadores basados en Chrome en modo headless. Pyppeteer ofrece un entorno ligero, rápido y centrado en tareas como la renderización y la captura, lo que la hace ideal para flujos automatizados de reconocimiento.

Durante el proceso de evaluación también se consideró el uso de **Selenium** [42], una herramienta muy popular para la automatización de navegadores. Sin embargo, fue descartada debido a su mayor peso y a que está más orientada a pruebas funcionales complejas, lo cual excede las necesidades de esta fase.

La Tabla 4.9 muestra una comparativa entre ambas soluciones según los criterios definidos.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
Pyppeteer	★★☆	★★★	★★★	★★☆	★★☆	★★★
Selenium	★★★	★★★	★☆☆	★★★	★★★	★★★

Tabla 4.9: Comparativa de herramientas para la captura automatizada de sitios web.

Se optó por utilizar **Pyppeteer** como herramienta principal para la captura de pantalla del sitio web debido a su mejor rendimiento para realizar esta tarea comparado

con **Selenium**.

Enumeración de directorios y archivos web

Durante esta fase se busca identificar rutas ocultas, recursos sensibles y directorios accesibles en la estructura web del objetivo. Para ello se ha seleccionado **Feroxbuster** [43], una herramienta moderna, con alto rendimiento, recursividad y excelente manejo de respuestas HTTP y redirecciones. Su salida estructurada y su integración mediante CLI la hacen especialmente adecuada para procesos automatizados.

Se evaluaron otras alternativas que ofrecían funcionalidades similares:

- **Gobuster** [44]: una de las herramientas más conocidas, desarrollada en Go. Aunque eficiente, no ofrece soporte de recursividad automática y su rendimiento es inferior al de otras opciones más modernas. Además, es una alternativa más obsoleta en comparación con el resto de herramientas.
- **Dirsearch** [45]: herramienta desarrollada en Python, altamente configurable y en constante mantenimiento. Ofrece unas prestaciones similares a las de **Feroxbuster**.
- **FFUF** [46]: herramienta muy potente y veloz desarrollada en Go, con salida en JSON, ideal para tareas de fuzzing de directorios y parámetros. Aunque comparable en prestaciones, se priorizó **Feroxbuster** por su mayor enfoque a la enumeración recursiva de rutas y mayor adopción en entornos *Red Team* (seguridad ofensiva).

La Tabla 4.10 muestra la comparativa de estas herramientas según los criterios establecidos.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
Feroxbuster	★★★	★★★	★★★	★★★	★★★	★★★
FFUF	★★★	★★★	★★★	★★★	★★☆	★★★
Dirsearch	★★★	★★★	★★★	★★☆	★★★	★★★
Gobuster	★★★	★★★	★★☆	★☆☆	★☆☆	★★★

Tabla 4.10: Comparativa de herramientas para enumeración de directorios web.

Por todo lo anterior, se optó por **Feroxbuster** como la herramienta principal para la enumeración de directorios y archivos web. Aunque tiene prestaciones muy similares a **Dirsearch**, se eligió por ofrecer mejores resultados en las pruebas de rendimiento realizadas.

Reconocimiento WordPress

En esta fase se analiza el CMS detectado previamente. Si se trata de un sitio **WordPress**, se activa un módulo específico que emplea la herramienta **WPScan** [47].

Se utiliza ésta para llevar a cabo un reconocimiento avanzado del sitio WordPress, incluyendo:

- **Enumeración de usuarios**: los usuarios enumerados se almacenan en una *wordlist* para su uso en fases posteriores, como ataques de fuerza bruta.
- **Enumeración de *directory listing***: se identifican rutas que permiten listar directorios y archivos, lo que puede revelar información sensible o configuraciones expuestas.
- **Enumeración de plugins**: los plugins instalados se analizan para identificar posibles vulnerabilidades (CVEs). Si se encuentra un exploit (código que aprovecha una vulnerabilidad) público asociado en GitHub, se clona automáticamente para facilitar su análisis posterior.

En caso de que se hayan identificado usuarios, se lanza un ataque de fuerza bruta mediante el propio **WPScan**, aprovechando el recurso `xmlrpc.php` [48], que permite realizar múltiples intentos de autenticación sin restricciones. Si no se ha podido enumerar ningún usuario, se utiliza una lista por defecto con credenciales comunes.

Se evaluó el panorama de herramientas disponibles para reconocimiento específico de sitios WordPress. Tras este estudio, se concluyó que **WPScan** no tiene actualmente una competencia directa que ofrezca un conjunto de funcionalidades tan completo y actualizado. Es, por tanto, la opción más sólida y especializada para llevar a cabo esta fase.

Reconocimiento Joomla

Si el CMS detectado es **Joomla**, se activa un módulo específico que emplea la herramienta **JoomScan** [49]. Esta herramienta, desarrollada por la OWASP Foundation, permite llevar a cabo un análisis específico de sitios Joomla, identificando vulnerabilidades conocidas y configuraciones inseguras.

Entre las funcionalidades más relevantes que realiza este módulo se incluyen:

- Detección de la versión del CMS y comprobación de vulnerabilidades asociadas.
- Identificación de archivos de configuración expuestos y copias de seguridad accesibles.
- Análisis de configuraciones comunes que pueden representar un riesgo de seguridad.

Al igual que en el caso de WPScan, se optó por **JoomScan** debido a su especialización en este CMS y a la falta de herramientas equivalentes que ofrezcan un conjunto de funcionalidades tan completo y específico.

Enumeración del servicio FTP

En caso de que el puerto 21 se encuentre abierto en el objetivo, la herramienta activa un módulo específico para la enumeración del servicio **FTP**. Esta fase tiene como objetivo identificar posibles accesos abiertos, credenciales débiles o mal configuradas, así como contenido sensible accesible desde el servidor.

Para ello, se emplea la librería de Python **ftplib** [50]. Ésta permite una interacción directa con el servicio FTP de forma sencilla y eficiente. Con ella se realizan las siguientes acciones:

- Se comprueba si el servidor permite acceso mediante login anónimo.
- Se realiza un ataque de fuerza bruta con credenciales comunes o personalizadas, en caso de que se hayan enumerado usuarios previamente.
- Se verifican permisos de lectura y escritura, en caso de acceso exitoso.
- Descarga automatizada del contenido almacenado en el servidor, si es accesible.

Durante el estudio de diferentes alternativas no se identificaron herramientas similares a **ftplib** que ofrecieran el mismo nivel de integración y simplicidad para tareas de automatización. Su inclusión directa en el ecosistema de Python y su facilidad de uso la convierten en la opción más adecuada para esta fase del reconocimiento.

Además, se realiza un análisis de la versión del servicio FTP con el objetivo de detectar vulnerabilidades conocidas. Para ello se utiliza **SearchSploit** [51], una herramienta ampliamente reconocida en el ámbito del pentesting, que permite buscar exploits públicos y pruebas de concepto asociadas a versiones concretas de software.

No se contemplaron herramientas alternativas a **SearchSploit**, ya que constituye un estándar de facto para la consulta de vulnerabilidades, especialmente útil en contextos donde se busca una solución rápida.

Enumeración del servicio SSH

En caso de que el puerto 22 (SSH) se encuentre accesible, la herramienta activa un módulo específico para el reconocimiento de este servicio. Para ello, se ha seleccionado la herramienta **Hydra** [52], ampliamente reconocida en el ámbito del pentesting por su flexibilidad y soporte para múltiples protocolos.

Inicialmente se consideró el uso de **Paramiko** [53], una librería de Python orientada a la gestión de conexiones SSH. No obstante, fue descartada tras comprobar que no está diseñada para realizar ataques de fuerza bruta. Su arquitectura orientada a sesiones estables y autenticación segura generaba errores al lanzar múltiples intentos de login consecutivos.

La Tabla 4.11 presenta la comparativa entre ambas soluciones evaluadas.

Herramienta	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
Hydra	★★★	★★☆	★★☆	★★☆	★★☆	★★☆
Paramiko	★☆☆	★★★	★☆☆	★★☆	★★☆	★★★

Tabla 4.11: Comparativa de herramientas para fuerza bruta SSH.

Por tratarse de un estándar de facto en ataques de fuerza bruta a servicios de red, se optó finalmente por **Hydra**.

Adicionalmente, la versión del servicio SSH detectada se analiza mediante **SearchSploit**, con el objetivo de identificar vulnerabilidades públicas asociadas y comprobar si existen exploits conocidos que puedan ser aprovechados.

Generación de informes

Para la generación de informes se utiliza la librería de Python **ReportLab** [54], que permite crear documentos PDF desde cero con un alto grado de personalización y control

sobre el diseño del contenido. Esta funcionalidad resulta clave en el contexto del proyecto, ya que los informes generados deben incluir texto estructurado, tablas, imágenes (como capturas de pantalla) y separadores visuales entre secciones.

Se consideró también la opción de utilizar la librería **fpdf2** [55], una versión moderna y mejorada del clásico FPDF, orientada a la generación de documentos PDF de forma sencilla y estructurada.

Ambas herramientas ofrecen un enfoque programático desde Python, sin necesidad de plantillas HTML externas, y permiten trabajar con texto, estilos, tablas e imágenes. Sin embargo, existen diferencias significativas en cuanto al nivel de control y flexibilidad que ofrecen.

La Tabla 4.12 muestra una comparativa entre ambas librerías según los criterios establecidos.

Librería	Reconocimiento	Integración	Rendimiento	Actualización	Flexibilidad	Documentación
ReportLab	★★★	★★★	★★★	★★★	★★★	★★★
fpdf2	★★☆	★★★	★★★	★★★	★★☆	★★★

Tabla 4.12: Comparativa de herramientas para generación de informes PDF.

Finalmente, se optó por **ReportLab** como la solución más adecuada debido a su reconocimiento en la comunidad, mayor control sobre el diseño y capacidad para generar documentos más complejos con mayor facilidad, especialmente al integrar elementos visuales como capturas de pantalla.

Asistente de inteligencia artificial

La herramienta incorpora un agente de asistencia basado en inteligencia artificial, diseñado para complementar el análisis técnico mediante la generación de recomendaciones y respuestas contextualizadas a partir de los resultados del escaneo. Este asistente se basa

en un modelo generativo de lenguaje natural y permite al usuario:

- **Consultar recomendaciones o dudas**: el usuario puede plantear preguntas sobre el análisis realizado, como vulnerabilidades detectadas, resultados obtenidos o interpretación de configuraciones.
- **Planificar próximos pasos**: el modelo sugiere acciones futuras en función del contexto del objetivo y del tipo de debilidades encontradas.
- **Proponer soluciones técnicas**: el asistente ofrece alternativas para mitigar o resolver los problemas de seguridad identificados.

Para implementar esta funcionalidad se evaluaron diversas opciones de modelos de lenguaje, priorizando aquellos que ofrecieran una API con capa gratuita. Entre las alternativas consideradas se encontraban:

- **Gemini (Google)** [56]: modelo de lenguaje avanzado con una API gratuita muy completa, ideal para el contexto del proyecto.
- **Mistral** [57]: ofrece diferentes modelos muy completos, pero la capa gratuita de su API es mucho más limitada que la de **Gemini**.
- **GPT-4 / GPT-3.5 (OpenAI)** [58]: modelos de referencia en la industria por su precisión y potencia, pero sin capa gratuita en su API oficial.
- **DeepSeek** [59]: plataforma emergente con modelos avanzados, pero sin plan gratuito en su API.

Finalmente se optó por usar **Gemini**, ya que, junto con **Mistral**, eran las únicas alternativas que contaban con una API con capa gratuita. Sin embargo, la de **Mistral** es mucho más limitada, con un máximo de 128.000 tokens frente a los 1.000.000 de **Gemini**.

En cuanto al modelo, se optó por integrar **Gemini Flash 2.0** debido a que:

- Ofrece una **capa gratuita**, sin necesidad de suscripción comercial.
- Soporta hasta **1.000.000 de tokens por minuto (TPM)**, lo que permite múltiples interacciones sin cuellos de botella.
- Permite hasta **1.500 solicitudes por día (RPD)** y **15 solicitudes por minuto (RPM)**, más que suficiente para el contexto del proyecto.
- Toda la generación (input/output) es **gratuita**, lo que elimina barreras de acceso económico.

El modelo no incluye integración con *Google Search*, y como es habitual en este tipo de servicios, los datos enviados pueden ser utilizados para mejorar el modelo de forma anónima.

Por su equilibrio entre capacidad, coste cero, integración sencilla y rendimiento, **Gemini Flash 2.0** se posiciona como la opción ideal para el asistente inteligente de la herramienta.

4.3. Resumen de herramientas utilizadas

A continuación se presenta un resumen de las herramientas utilizadas en la propuesta.

1. **Escaneo de puertos y servicios**: `Nmap`
2. **Reconocimiento DNS**: `DNSdumpster`, `MXToolbox` y `API Ninja (WHOIS)`
3. **Email OSINT**: `IntelligenceX` y `LeakCheck`
4. **Enumeración de subdominios**: `Subfinder` y `HTTPX`
5. **Escaneo de vulnerabilidades web**: `Nuclei`
6. **Detección de CMS**: `WhatWeb`

7. **Captura de pantalla web**: `Pyppeteer`
8. **Enumeración de directorios y archivos web**: `Feroxbuster`
9. **Reconocimiento WordPress**: `WPScan`
10. **Reconocimiento Joomla**: `Joomscan`
11. **Reconocimiento FTP**: `ftplib` y `Searchsploit`
12. **Reconocimiento SSH**: `Hydra` y `Searchsploit`
13. **Generación de informe**: `ReportLab`
14. **Asistente de IA**: `Gemini Flash 2.0`

La Figura 4.1 muestra un esquema visual con el resumen de las herramientas utilizadas en cada fase:

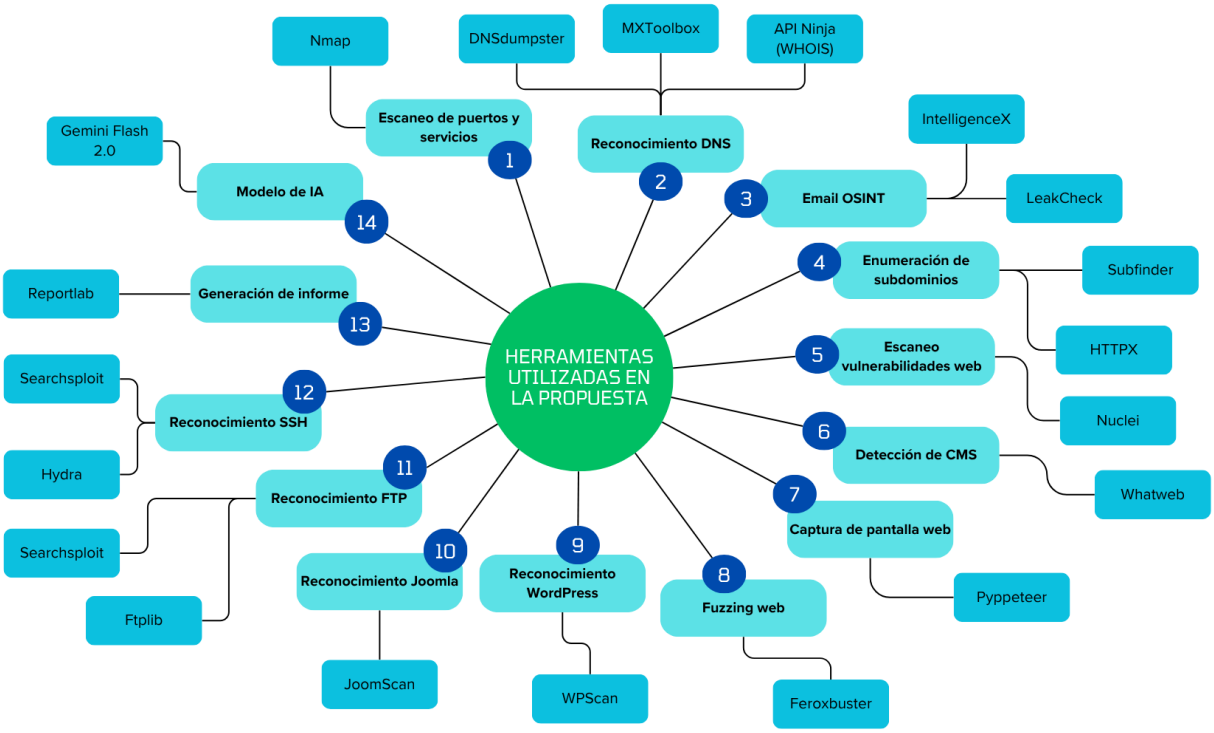


Figura 4.1: Figura resumen de las herramientas utilizadas en la propuesta.

4.4. Flujo metodológico de ejecución

El flujo metodológico seguido por la herramienta desarrollada se estructura en distintas fases que se ejecutan de forma secuencial o condicional, en función de las características del objetivo introducido y de los servicios detectados durante el escaneo. A continuación, se describen cada una de estas fases.

4.4.1. Escaneo de puertos y servicios

Se inicia con la ejecución de `Nmap`, con el objetivo de identificar los puertos abiertos del objetivo, los servicios en ejecución sobre dichos puertos y sus versiones.

4.4.2. Enumeración del dominio

En caso de que el objetivo sea un dominio (y no una dirección IP), se lanzan tres fases específicas.

Reconocimiento DNS

Se realiza un reconocimiento pasivo del dominio, recopilando información pública y registros DNS. Se utilizan varias herramientas para llevar a cabo esta tarea:

- Consulta de registros DNS mediante `DNSdumpster`, obteniendo información como registros `MX`, `NS` y `TXT`.
- Análisis del registro `DMARC` a través de `MXToolbox`, útil para detectar posibles vectores de ataques relacionados con *email spoofing*.
- Consulta WHOIS utilizando la API de `API Ninja`, lo que permite recuperar datos como propietario del dominio, fechas de creación y expiración, y otra metainformación.

OSINT de direcciones de correo electrónico

Se extraen direcciones asociadas al dominio a través de `IntelligenceX`, y posteriormente se verifica si han sido expuestas en brechas de seguridad mediante `LeakCheck`.

Enumeración de subdominios

Se utiliza `Subfinder` para obtener subdominios vinculados al dominio objetivo. Posteriormente, se emplea `HTTPX` para validar cuáles de ellos están activos, comprobando el código de estado HTTP.

4.4.3. Reconocimiento web

Si se detecta que el puerto 80 está abierto, se lanzan una serie de modulos para realizar un reconocimiento exhaustivo del servicio web.

Escaneo de vulnerabilidades web

Se lanza `Nuclei` para buscar vulnerabilidades en la aplicación web. Si se activa el modo de *escaneo exhaustivo*, también se escanean todos los subdominios activos descubiertos.

Detección del CMS

Se ejecuta `WhatWeb` para identificar si el sitio utiliza algún gestor de contenidos (WordPress, Joomla, etc.).

Captura de pantalla

Se genera una captura visual del sitio web utilizando `Pyppeteer`, la cual será incluida en el informe final.

Enumeración de directorios y archivos

Se utiliza `Feroxbuster` para descubrir rutas y archivos accesibles. En modo exhaustivo se lanza con recursividad de nivel 2, mientras que en modo normal se limita a nivel 1.

Reconocimiento WordPress

Si el CMS detectado es WordPress, se emplea `WPScan` para:

- Enumerar usuarios y plugins instalados.
- Consultar posibles CVEs asociados a los plugins.
- Descargar automáticamente exploits si están disponibles públicamente en GitHub.

Fuerza bruta en WordPress

Se lanza un ataque sobre `xmlrpc.php` mediante `WPScan`. Se utilizan las credenciales enumeradas en la fase anterior o, en su defecto, una lista por defecto con credenciales comunes.

Reconocimiento Joomla

Si se detecta Joomla, se utiliza `Joomscan` para identificar vulnerabilidades específicas de este CMS, configuraciones inseguras, copias de seguridad accesibles o archivos sensibles.

4.4.4. Reconocimiento FTP

Si el puerto 21 está abierto, se utiliza `ftplib` para:

- Verificar si el login anónimo está habilitado.
- Realizar fuerza bruta con wordlists (personalizadas o por defecto).
- Comprobar permisos de lectura/escritura y descargar archivos.
- Consultar vulnerabilidades conocidas mediante `Searchsploit`.

4.4.5. Reconocimiento SSH

Si el puerto 22 está abierto, se utiliza `Hydra` para realizar un ataque de fuerza bruta, utilizando las credenciales obtenidas anteriormente o una wordlist genérica. También se consulta `Searchsploit` para comprobar vulnerabilidades de versión.

4.4.6. Generación del informe

Finalmente, se genera un informe PDF estructurado con todos los resultados obtenidos, incluyendo capturas, tablas y secciones organizadas por fase. En caso de que el escaneo se haya realizado dentro de un proyecto, el informe se versiona automáticamente, permitiendo mantener un histórico de auditorías por objetivo.

4.5. Figura resumen del flujo de ejecución

La Figura 4.2 muestra un esquema general del flujo metodológico que sigue la herramienta desarrollada.

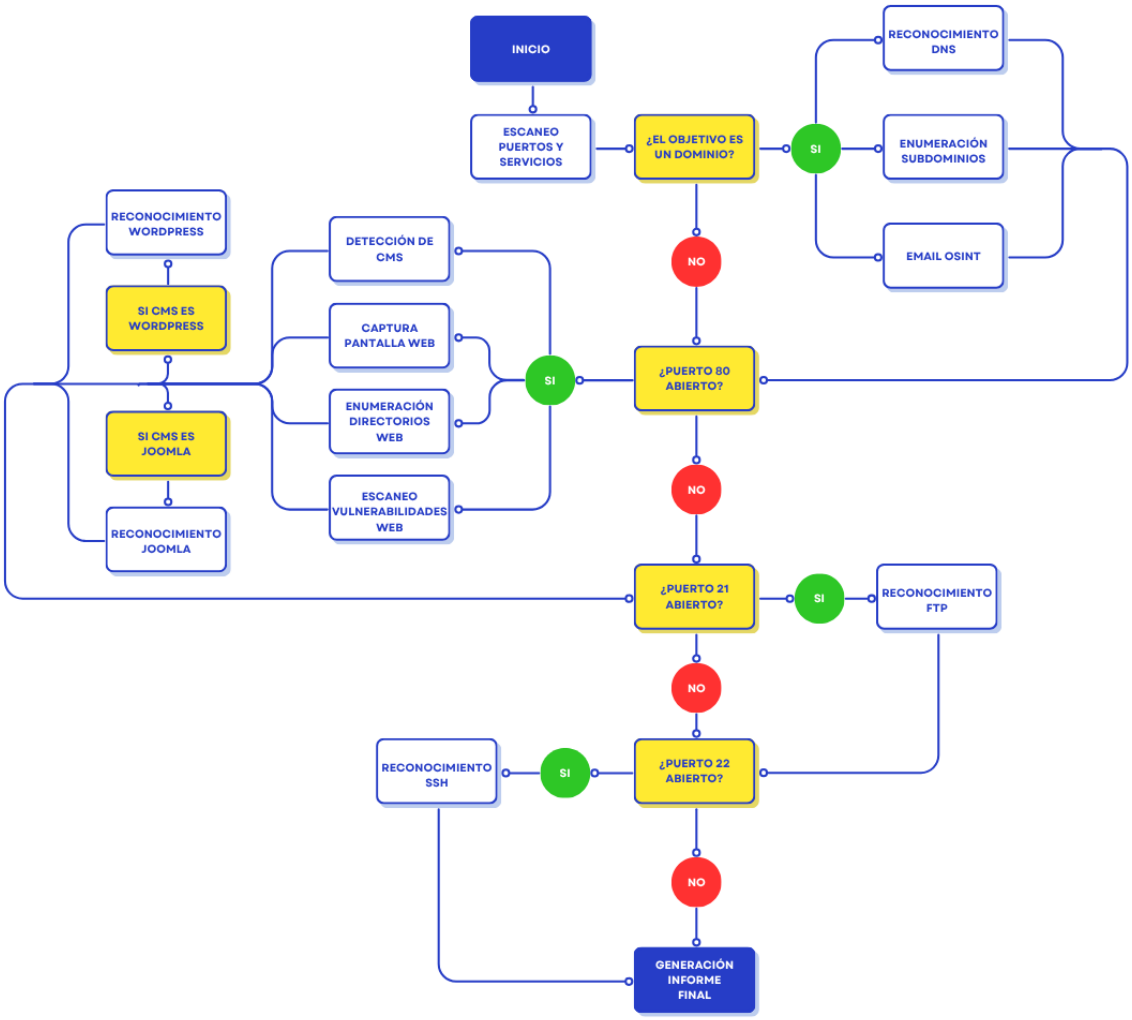


Figura 4.2: Flujo metodológico de ejecución de la herramienta.

Capítulo 5

Implementación y desarrollo

Contenidos

5.1. Diseño de la arquitectura de la herramienta 52

5.1.1. Tecnologías empleadas 57

5.1.2. Comunicación entre componentes 59

5.2. Decisiones técnicas 60

5.2.1. Frontend: HTML, CSS y JavaScript 60

5.2.2. Backend: Python y Flask 61

5.2.3. Orquestación mediante `main.py` 62

5.3. Diseño experimental 62

5.3.1. Requisitos previos 62

5.3.2. Configuración del escaneo 64

5.3.3. Flujo de uso de la herramienta 71

5.1. Diseño de la arquitectura de la herramienta

La herramienta ha sido diseñada con un enfoque modular, estructurada en distintos componentes que colaboran entre sí. Esta arquitectura permite mantener un flujo de ejecución claro, así como facilitar futuras ampliaciones, modificaciones o mantenimientos del sistema.

El núcleo principal del sistema es el archivo **main.py**, encargado de orquestar todo el proceso de escaneo. Este script actúa como controlador central, invocando los módulos individuales contenidos en la carpeta `/modules`, cada uno de los cuales implementa una fase concreta del reconocimiento. El flujo general está guiado por condiciones lógicas que determinan qué módulos deben ejecutarse en función de las características del objetivo.

Cada uno de estos módulos está implementado como un archivo Python independiente. Éstos siguen una estructura similar en la cual se definen funciones internas auxiliares para realizar tareas concretas, y al final del archivo se incluye una función con el nombre `run_nombre_del_módulo()`, que es la encargada de ejecutar el módulo completo y es invocada desde el `main.py`. Estos scripts almacenan sus resultados en la carpeta `/logs`, los cuales serán utilizados posteriormente para generar el informe. A modo de ejemplo se muestra el pseudocódigo del módulo `nmap.py`:

Listing 5.1: Estructura general de un módulo

```
# modules/nmap.py
def initial_scan(target_ip):
    # Escaneo inicial de puertos
    ...

def extract_ports(target_ip):
    # Extrae los puertos abiertos
    ...

```

```
def hard_scan(target_ip, open_ports):
    # Escaneo profundo en los puertos abiertos
    ...

def clear_report(target_ip):
    # Guarda los resultados en /logs
    ...

def run_nmap(target_ip):
    print("[*] Running nmap module...")
    initial_scan(target_ip)
    open_ports = extract_ports(target_ip)
    hard_scan(target_ip, open_ports)
    clear_report(target_ip)
    print("[+] Nmap module completed.")
```

La lógica del flujo de escaneo implementada en `main.py` puede resumirse en el siguiente pseudocódigo:

Listing 5.2: Pseudocódigo del archivo *main.py*

```
# Escaneo inicial
run_nmap(target)

# Si el objetivo no es una IP, se trata de un dominio
try:
    ipaddress.ip_address(target)
except ValueError:
    run_dns_recon(target)

# Condiciones en funcion de los puertos detectados
```

```
11 if port_80_open:
12     run_web_modules(target)
13
14 if detected_cms == "wordpress":
15     run_wordpress(target)
16
17 if detected_cms == "joomla":
18     run_joomla(target)
19
20 if port_21_open:
21     run_ftp(target)
22
23 if port_22_open:
24     run_ssh(target)
25
26 # Generacion del informe final
27 run_generate_report(target, flag)
```

Por otra parte, el archivo **app.py** actúa como servidor web, encargado de gestionar la interfaz gráfica. Este archivo implementa una API interna mediante Flask, que conecta la interfaz (cliente) con el backend (lanzando llamadas al **main.py** y otras funciones auxiliares). A través de esta API se pueden lanzar escaneos, interactuar con el modelo de inteligencia artificial, consultar resultados anteriores o generar informes.

La interfaz gráfica está implementada como una aplicación de una sola página (*Single Page Application*). Se compone principalmente de un archivo HTML (**index.html**) y un archivo JavaScript (**script.js**) que gestiona la interacción dinámica del usuario. Este último se encarga de manipular los elementos del DOM (*Document Object Model*) [60], y realizar peticiones `fetch` a los endpoints de **app.py**. Por ejemplo, cuando el usuario pulsa el botón de "Iniciar Escaneo", se ejecuta una petición `fetch(/fullscan)` enviando

los parámetros seleccionados por el usuario para lanzar el análisis completo.

La integración con el modelo de Gemini se realiza utilizando la librería de Python `google.generativeai`. Se definen tres endpoints clave:

- `/chat`: permite enviar preguntas al modelo IA desde el chat integrado.
- `/next_offensive`: genera automáticamente próximos pasos ofensivos a seguir.
- `/next_defensive`: genera automáticamente recomendaciones defensivas.

Todos los endpoints siguen una estructura similar: reciben la petición, cargan el contexto del informe más reciente, construyen el prompt y obtienen la respuesta del modelo. La diferencia entre ellos radica principalmente en el prompt utilizado. En el caso de `/chat`, se envía la pregunta del usuario junto con el informe, mientras que en los otros dos no es necesario incluir la pregunta, ya que el prompt es fijo. El siguiente pseudocódigo muestra la estructura general de estos endpoints:

Listing 5.3: Estructura general de los endpoints de IA

```
@app.route("/endpoint", methods=["POST"])
def ia_endpoint():
    # Configurar API token
    google.generativeai.configure(api_key=API_KEY)

    # Cargar el contexto del informe más reciente
    report = load_latest_report()

    # Construir el prompt para la IA
    prompt = f"Informe: {report}\n
               Pregunta: {request.json['question']}"

    # Llamar al modelo de IA
    model = genai.GenerativeModel("gemini-2.0-flash")
```

```
    response = model.generate_content(prompt)

    # Devolver la respuesta al cliente (endpoint /chat)
    return jsonify({"response": response.text})

    # Guardar la respuesta en md (endpoints
    # /next_offensive y /next_defensive)
    save_response_to_md(response.text)
```

La generación de informes se realiza mediante el módulo `generate_report.py`, el cual es invocado al final de `main.py`. Este módulo recoge todos los archivos de resultados generados por los submódulos (almacenados en `/logs`) y construye un documento PDF utilizando la librería `ReportLab`. La implementación de `generate_report.py` se resume en el siguiente pseudocódigo:

Listing 5.4: Pseudocódigo del archivo *generate_report.py*

```
# generate_report.py
def create_elements():
    # Crea los elementos del informe
    ...

def generate_report(target):
    # Crea el documento PDF con los datos guardados en /logs
    ...
    # Si bandera -p activa, se asocia a un proyecto
    ...
    doc = SimpleDocTemplate(path, pagesize=letter)
    for filename in os.listdir(logs_path):
        if filename.endswith(".txt"):
            write_to_pdf()
```

```
    ...

def run_generate_report(target, flag=None):
    generate_report(target, flag)
```

Esta arquitectura modular permite separar claramente la lógica de escaneo, la interfaz gráfica y la interacción con el modelo de IA, facilitando tanto el desarrollo como la depuración o sustitución de componentes individuales.

5.1.1. Tecnologías empleadas

El desarrollo de la herramienta se ha llevado a cabo mediante una combinación de tecnologías tanto en el apartado del frontend como del backend. A continuación, se describen las principales tecnologías utilizadas:

- **Frontend**: Se ha utilizado **HTML**, **CSS** y **JavaScript** para el diseño de la interfaz gráfica. JavaScript cumple una doble función: por un lado, permite gestionar la lógica de la interfaz (cambio de vistas, interacción dinámica con los elementos), y por otro, se encarga de establecer la comunicación con el backend mediante llamadas a la API, utilizando el método `fetch()`.
- **Backend**: Toda la lógica del backend está implementada en **Python**. Los módulos individuales encargados de realizar tareas específicas del reconocimiento están escritos en este lenguaje. Además, se utiliza **Flask**, un *framework* web desarrollado en Python, para construir el archivo `app.py`, el cual actúa como puente entre la interfaz gráfica y el núcleo del sistema. Este archivo expone los endpoints necesarios para que el frontend pueda lanzar escaneos, acceder a informes, gestionar proyectos o interactuar con el modelo de IA.
- **Módulos de escaneo**: Cada uno de los módulos del sistema implementa una

fase específica del proceso de auditoría y está escrito en Python. Estos módulos utilizan herramientas externas de línea de comandos o mediante APIs, como `Nmap`, `Subfinder`, `Nuclei`, `WPScan`, `Hydra`, entre otras, descritas en el capítulo de metodología.

- **Modelo de inteligencia artificial**: Para la parte de asistencia automatizada se ha integrado el modelo **Gemini Flash 2.0**, accedido a través de la API oficial proporcionada por Google. Las llamadas al modelo y el procesamiento de sus respuestas se realizan desde el backend en Python, integrándose de forma transparente en la interfaz para facilitar al usuario recomendaciones, próximos pasos o análisis contextualizado de los resultados.

La figura 5.1 resume las tecnologías empleadas en el desarrollo de la herramienta.

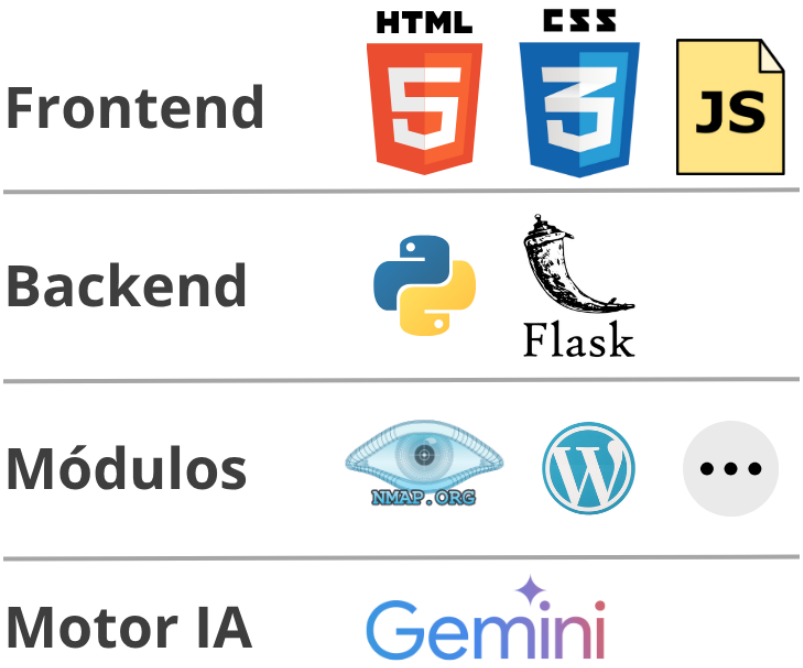


Figura 5.1: Tecnologías empleadas en la herramienta

5.1.2. Comunicación entre componentes

La herramienta ha sido diseñada siguiendo un modelo de arquitectura en el que cada componente tiene una función específica y se comunica con el resto de forma estructurada. Esta separación de responsabilidades permite una mayor mantenibilidad y escalabilidad, facilitando la incorporación de nuevas funcionalidades o la modificación de las existentes sin afectar al conjunto del sistema.

El flujo de comunicación entre los distintos componentes puede resumirse de la siguiente forma:

- El usuario interactúa con la interfaz gráfica.
- Cuando el usuario realiza una acción (por ejemplo, iniciar un escaneo), el archivo `script.js` lanza una petición `HTTP` mediante `fetch()` hacia un endpoint definido en `app.py`, el cual actúa como intermediario entre la interfaz y el backend.
- El archivo `app.py` procesa la solicitud recibida y lanza el script `main.py`, pasándole los parámetros necesarios (objetivo, modo de escaneo, etc.).
- El archivo `main.py` es el encargado de orquestar el proceso de escaneo completo. En función del tipo de objetivo y los puertos detectados, invoca secuencialmente los distintos módulos independientes ubicados en el directorio `/modules`, cada uno responsable de una fase concreta del reconocimiento.
- Los resultados generados por cada módulo se almacenan en archivos intermedios y son utilizados más adelante para generar un informe en formato PDF mediante la librería **ReportLab**.
- Por otro lado, cuando el usuario desea interactuar con el sistema de inteligencia artificial (por ejemplo, para solicitar recomendaciones o aclaraciones sobre el escaneo), la interfaz envía la petición a otro endpoint de `app.py`, que a su vez realiza una llamada a la **API de Gemini Flash 2.0**. La respuesta obtenida se devuelve a la interfaz y se muestra al usuario.

La Figura 5.2 resume gráficamente este flujo de comunicación y la relación entre los distintos bloques funcionales del sistema.

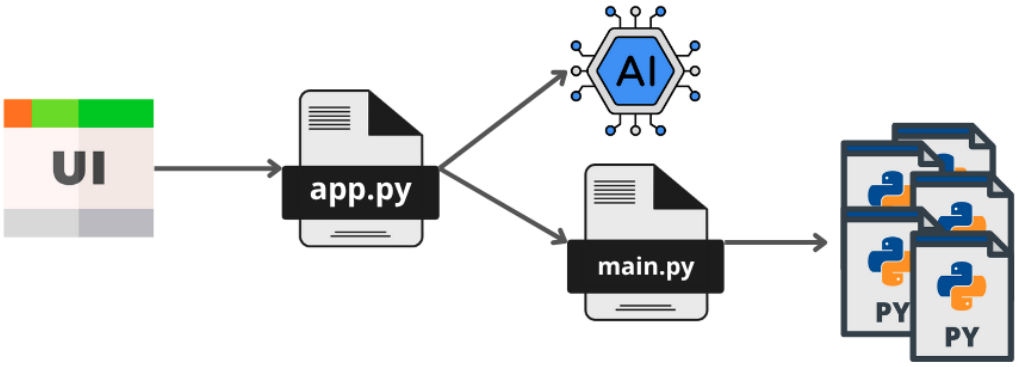


Figura 5.2: Flujo de comunicación entre los componentes de la herramienta.

Este diseño modular favorece la separación de responsabilidades y permite modificar o sustituir componentes concretos (como los módulos de escaneo o el motor de IA) sin afectar al resto de la aplicación, facilitando así su mantenimiento y evolución futura.

5.2. Decisiones técnicas

Durante el desarrollo de la herramienta se han tomado una serie de decisiones técnicas orientadas a maximizar la simplicidad, la eficiencia y la facilidad de mantenimiento del sistema. A continuación, se justifican las elecciones más relevantes respecto a tecnologías utilizadas y otras que fueron descartadas.

5.2.1. Frontend: HTML, CSS y JavaScript

Se optó por una interfaz basada en HTML, CSS y JavaScript puro por las siguientes razones:

- **Simplicidad y ligereza:** al tratarse de una herramienta de propósito técnico, la interfaz no requiere grandes funcionalidades interactivas, por lo que el uso de tecnologías más avanzadas como React habría sido innecesario.
- **Facilidad de integración:** el frontend se comunica directamente con el backend mediante peticiones `fetch`, sin necesidad de routers, sesiones, ni gestión de estados complejos.
- **Rapidez de desarrollo:** la curva de aprendizaje es más baja y permite una implementación más directa y rápida.

Tecnologías descartadas: React (incluyendo Next.js) fue considerado inicialmente como posible alternativa. Sin embargo, su inclusión suponía una sobrecarga innecesaria en complejidad, ya que no se requerían características avanzadas como rutas dinámicas, autenticación, manejo de estado o conexión con bases de datos. Por ello, se optó por una solución más ligera y directa.

5.2.2. Backend: Python y Flask

La elección de Python como lenguaje principal y de Flask como framework se justifica por:

- **Alta integración con herramientas de seguridad:** muchas de las utilidades utilizadas (Nmap, Subfinder, Nuclei, etc.) se invocan fácilmente desde Python mediante llamadas al sistema.
- **Desarrollo rápido y modular:** Flask proporciona una estructura clara, simple y fácil de mantener, ideal para proyectos de esta naturaleza.
- **Comunicación efectiva con el frontend:** al actuar como backend de la interfaz, Flask permite exponer endpoints REST a los que se puede acceder desde JavaScript con facilidad.

5.2.3. Orquestación mediante `main.py`

Se decidió centralizar toda la lógica de ejecución de módulos en un único script llamado `main.py`, lo que permite:

- Encapsular la lógica del flujo de ejecución.
- Garantizar la reutilización del código tanto desde la consola como desde la interfaz.
- Permitir una estructura modular, desacoplando cada funcionalidad en archivos separados dentro del directorio `/modules`.

En conjunto, estas tecnologías han permitido desarrollar una herramienta modular y fácilmente extensible, sin recurrir a opciones innecesariamente complejas para el propósito de la misma.

5.3. Diseño experimental

Esta sección describe los pasos necesarios para realizar una auditoría utilizando la herramienta desarrollada.

5.3.1. Requisitos previos

Antes de ejecutar la herramienta, es necesario cumplir con una serie de requisitos previos, tanto a nivel de software como de red.

- **Entorno de ejecución**: La herramienta ha sido diseñada y probada sobre sistemas operativos GNU/Linux, por lo que se recomienda su uso en distribuciones como Ubuntu, Debian o Kali Linux.

- **Python 3**: Es imprescindible contar con una versión de Python 3 instalada en el sistema. Se recomienda Python 3.8 o superior para asegurar la compatibilidad con todas las dependencias.

- **Instalación de dependencias**: Las librerías necesarias para ejecutar la herramienta se encuentran especificadas en el archivo `requirements.txt`. Se pueden instalar con el siguiente comando: `pip install -r requirements.txt`

- **Herramientas externas**: Algunas funcionalidades dependen de herramientas externas que deben instalarse manualmente en el sistema. Estas incluyen:

  - **Nmap**
  - **Subfinder**
  - **Httpx**
  - **Nuclei**
  - **WhatWeb**
  - **Feroxbuster**
  - **WPScan**
  - **Joomscan**
  - **Hydra**
  - **Searchsploit**

  Es recomendable añadir estas herramientas al `PATH` del sistema para que puedan ser invocadas desde cualquier módulo.

- **Acceso a Internet**: La herramienta requiere conexión a Internet para funcionalidades que se apoyan en APIs externas, como la consulta WHOIS, OSINT de correos electrónicos o interacción con el modelo de inteligencia artificial.

- **Claves de API**: Para utilizar algunos servicios, como API Ninja o IntelligenceX, es necesario disponer de una clave de API válida. Estas claves pueden obtenerse a través

de las páginas de cada servicio y añadirse desde la interfaz gráfica o directamente en el archivo de configuración correspondiente (en la carpeta `/config`).

5.3.2. Configuración del escaneo

La herramienta permite configurar distintos aspectos del escaneo, adaptándose así a múltiples escenarios y necesidades del auditor. A continuación, se describen las principales opciones de configuración disponibles:

- **Objetivo del escaneo**: el usuario puede especificar como objetivo tanto una dirección IP como un nombre de dominio o una URL. La herramienta detecta automáticamente el tipo de objetivo y ajusta el flujo de reconocimiento en consecuencia.
- **Modo de ejecución - interfaz gráfica o consola**:
  - *Interfaz gráfica (GUI)*: para lanzar un escaneo desde la interfaz, el usuario debe acceder a la pestaña de escaneo, introducir el objetivo y pulsar el botón `Start`. Los resultados se irán mostrando dinámicamente en la consola de escaneo integrada.

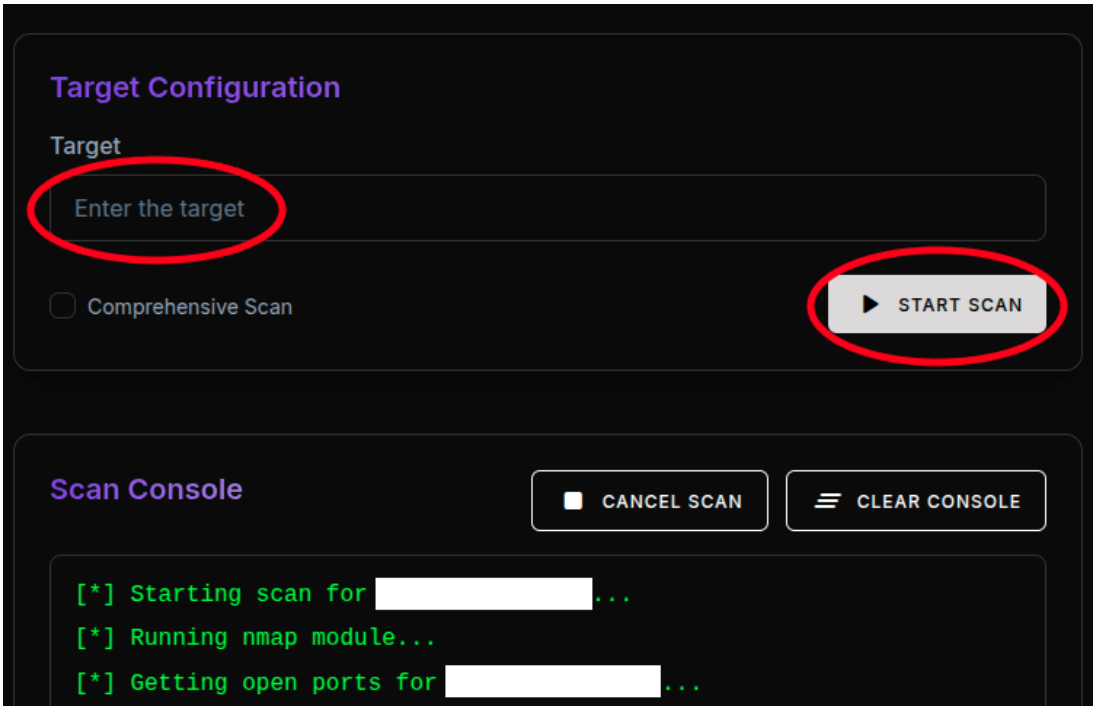


Figura 5.3: Escaneo desde la interfaz gráfica.

- *Consola (CLI)*: también es posible lanzar un escaneo directamente desde la línea de comandos. Para ello, basta con ejecutar:

```
python3 main.py <objetivo>
```

Figura 5.4: Escaneo desde consola.

- **Escaneo exhaustivo**: esta opción permite aumentar la profundidad del análisis. Al activarse:
  - Se lanza `Nuclei` para todos los subdominios activos, no solo para el dominio principal.
  - Se ejecuta `Feroxbuster` con nivel de recursividad 2 (en lugar del 1 por defecto).

Desde la interfaz gráfica, se puede activar esta opción marcando la casilla correspondiente al iniciar el escaneo. Desde consola, se activa con el parámetro `-ex`:

```
python3 main.py <objetivo> -ex
```

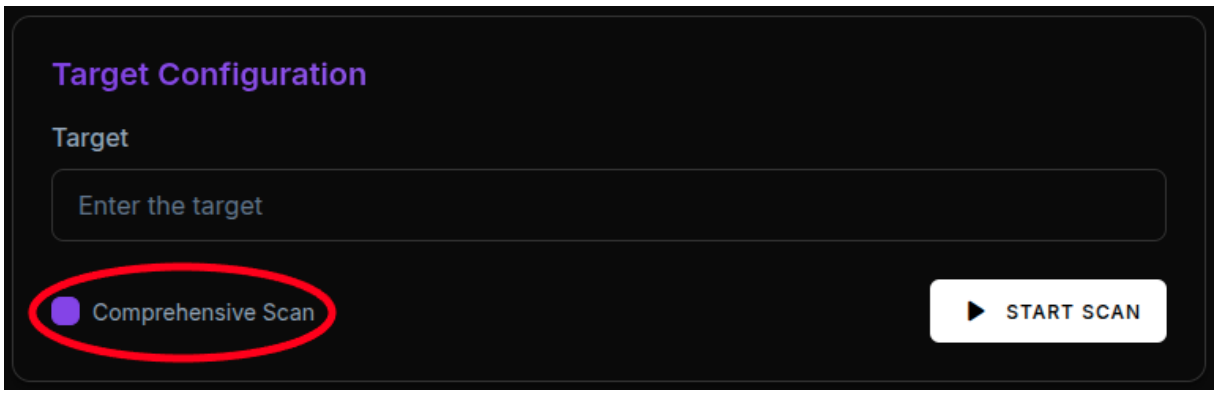


Figura 5.5: Escaneo exhaustivo desde la interfaz gráfica.

- **Asociación con proyecto**: una de las características principales de la herramienta es su sistema de proyectos, que permite agrupar los escaneos realizados sobre un mismo objetivo, gestionar versiones de informes y mantener un historial completo del análisis.
    - En la interfaz gráfica, los proyectos se crean desde la pestaña `Proyectos`. Allí se puede introducir el nombre del proyecto y el objetivo.

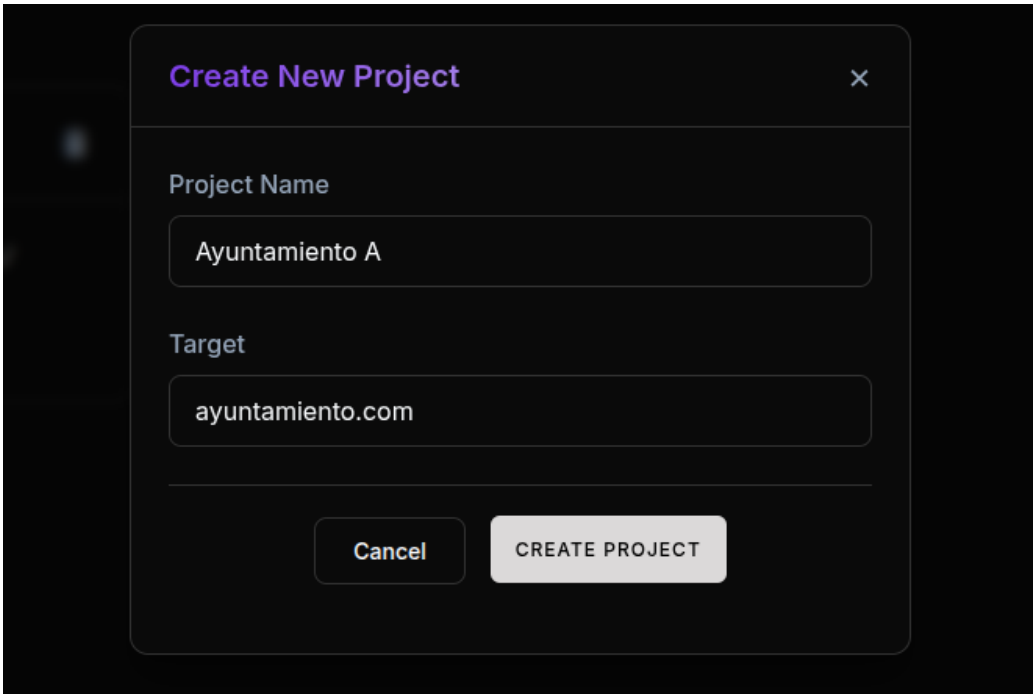


Figura 5.6: Formulario de creación de nuevo proyecto.

- Una vez creado, se accede a una vista específica del proyecto con tres pestañas:
  - **Consola de escaneo**: donde se lanza el reconocimiento y se visualiza el progreso.
  - **Historial de informes**: permite consultar los informes generados en auditorías anteriores.
  - **Chat con Orthrus**: chatbot integrado que utiliza inteligencia artificial para responder preguntas sobre el proyecto, sugerir próximos pasos o aclarar dudas sobre los resultados obtenidos.

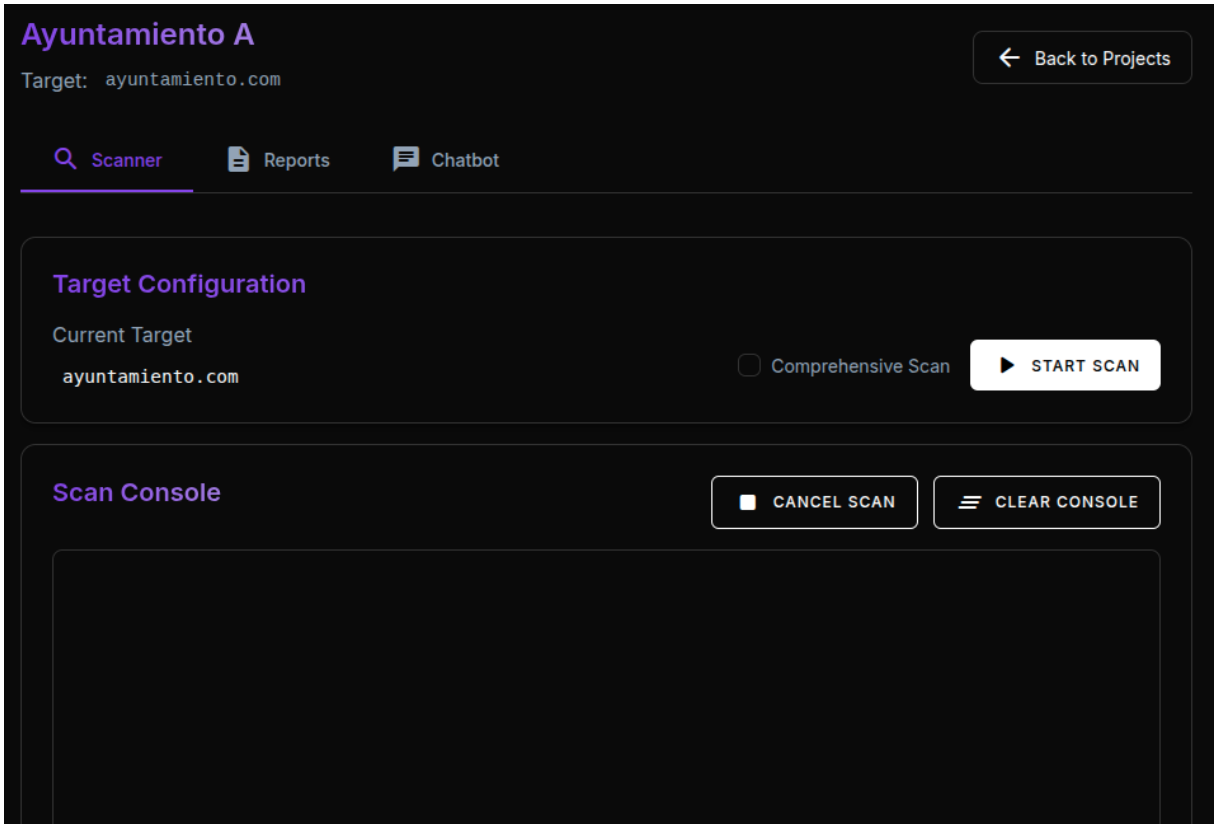


Figura 5.7: Consola de escaneo dentro de un proyecto.

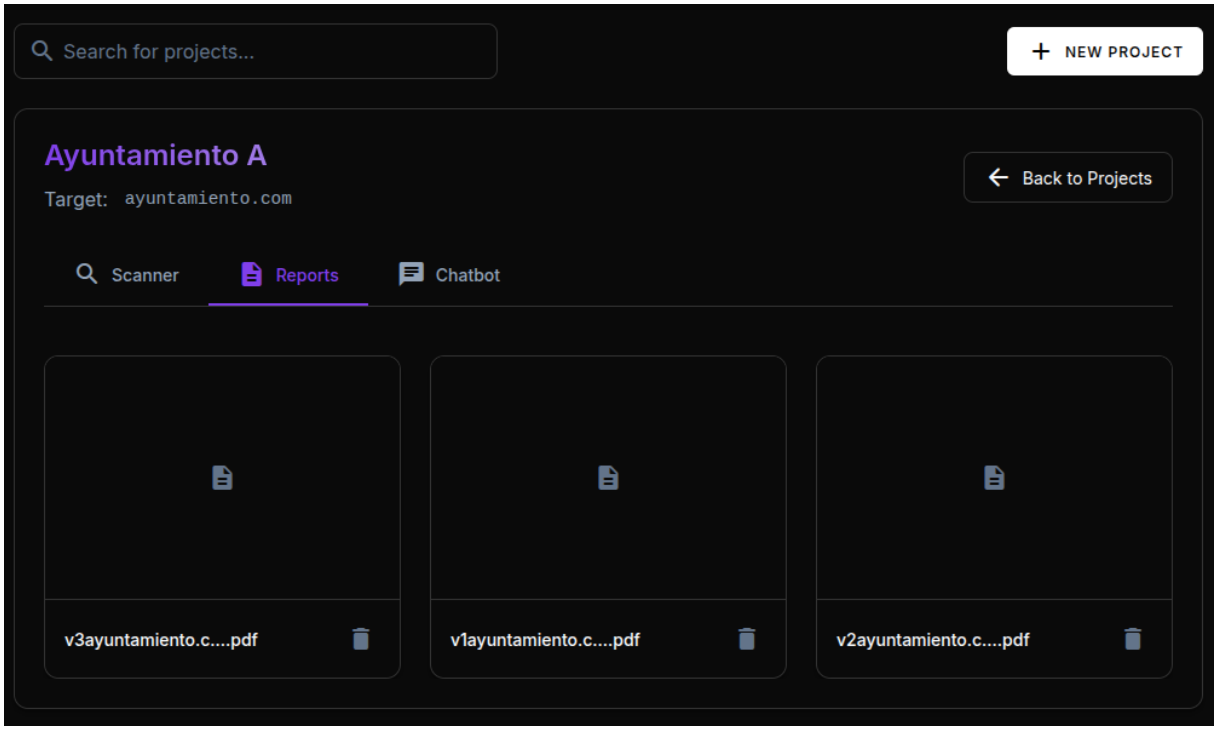


Figura 5.8: Historial de informes generados en un proyecto.

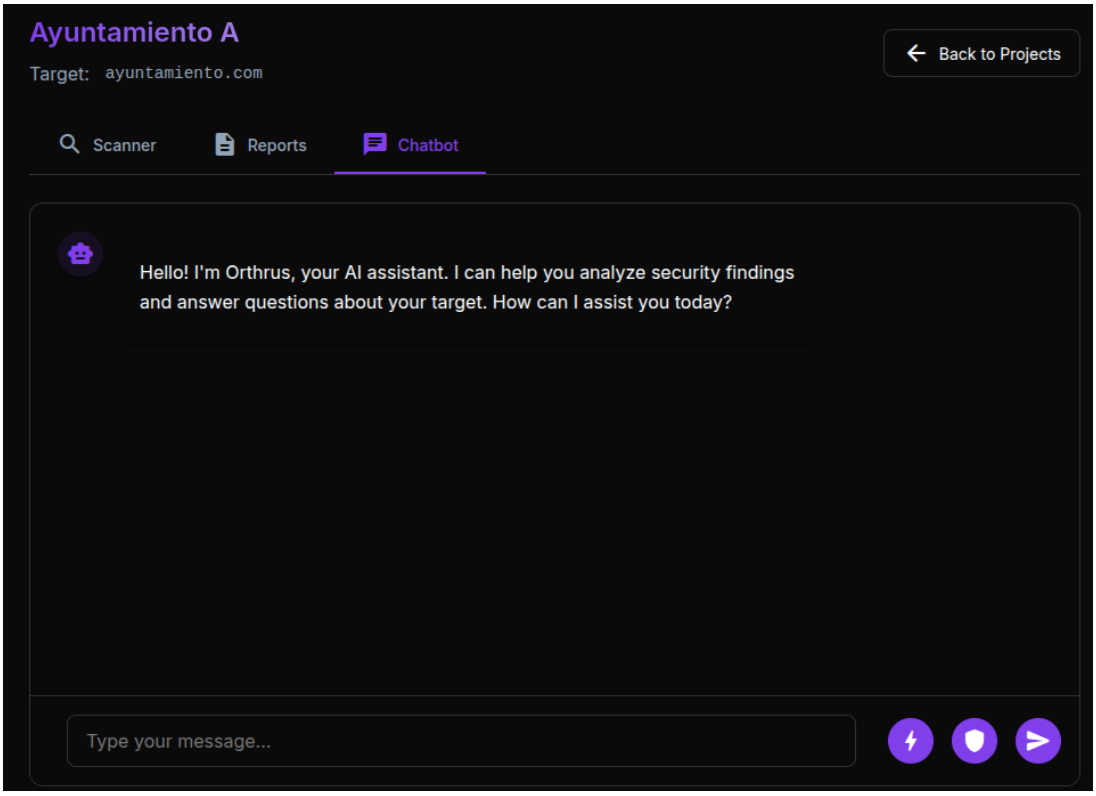


Figura 5.9: Interacción con el asistente de IA desde un proyecto.

- Desde consola, no es posible crear nuevos proyectos, pero sí asociar un escaneo a uno ya existente con el parámetro `-p`:

```
python3 main.py <objetivo> -p nombre_proyecto
```

- **Configuración de proxy**: desde la pestaña de configuración, se puede habilitar un proxy para los escaneos, especialmente útil para evitar bloqueos. La herramienta permite elegir entre proxies `HTTP` y `SOCKS5`. Este proxy se integra en `WPScan`, redirigiendo su tráfico por la ruta indicada.

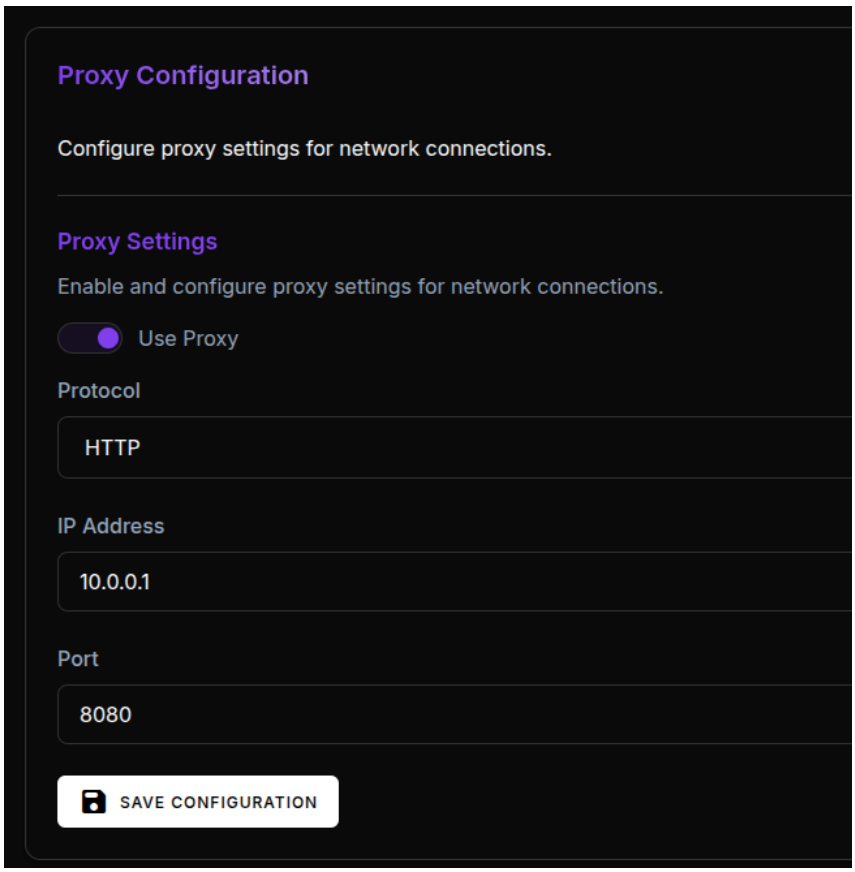


Figura 5.10: Configuración del proxy desde la interfaz.

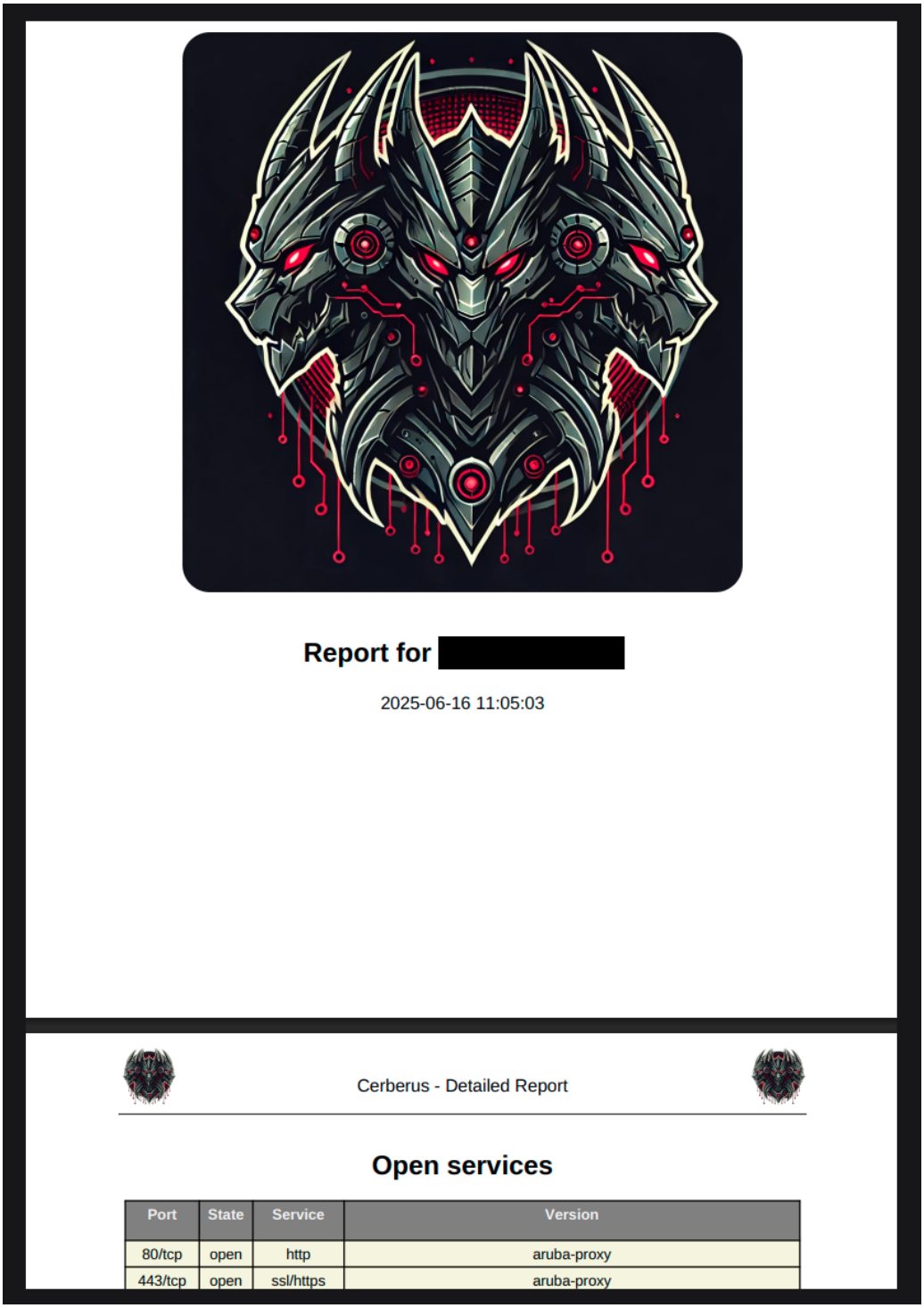
5.3.3. Flujo de uso de la herramienta

A continuación se describe un ejemplo del flujo completo de uso de la herramienta, aplicando la opción más realista y completa: la realización del escaneo desde un proyecto previamente creado en la interfaz gráfica.

El primer paso consiste en crear un nuevo proyecto asociado al objetivo, introduciendo los datos correspondientes. Este proceso se realiza desde la pestaña de "Proyectos", tal y como se ha mostrado anteriormente en la Figura 5.6.

Una vez creado el proyecto, se procede a lanzar el escaneo desde la consola de escaneo asociada al proyecto. Previamente, el usuario puede haber configurado opciones como el modo exhaustivo o el uso de un proxy. El lanzamiento del escaneo desde el entorno gráfico puede verse en la Figura 5.7, presentada en el apartado anterior.

Una vez finalizado el escaneo, la herramienta genera un informe PDF automáticamente, accesible desde la pestaña de informes del proyecto. Un ejemplo de este informe se muestra en la Figura 5.11.



Report for [redacted]

2025-06-16 11:05:03

Cerberus - Detailed Report

Open services

Port	State	Service	Version
80/tcp	open	http	aruba-proxy
443/tcp	open	ssl/https	aruba-proxy

Figura 5.11: Visualización del informe PDF generado.

A continuación, es habitual interactuar con la inteligencia artificial integrada en la herramienta. Esta interacción puede adoptar distintas formas:

- **Generación de siguientes pasos ofensivos:** el usuario puede pulsar el botón correspondiente para que la IA sugiera cómo continuar con la auditoría desde un enfoque ofensivo, tal y como se observa en la Figura 5.12.

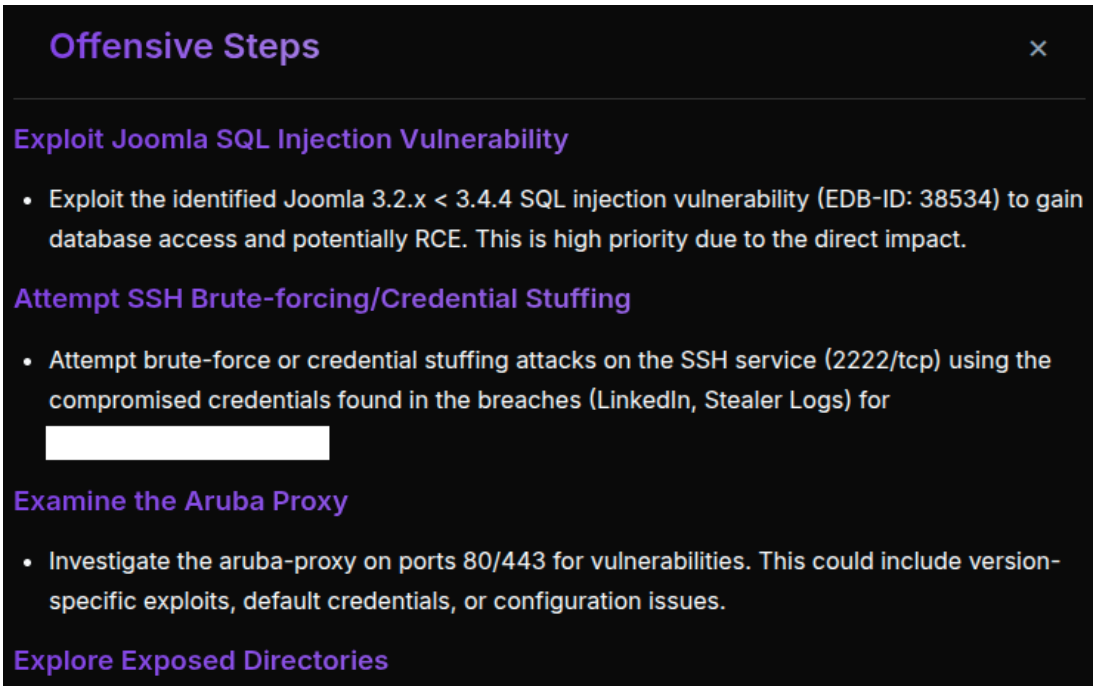


Figura 5.12: Generación de siguientes pasos desde una perspectiva ofensiva.

- **Generación de siguientes pasos defensivos:** por otra parte, se pueden generar recomendaciones defensivas para mitigar las debilidades detectadas, como se observa en la Figura 5.13.

Defensive Steps

Joomla Vulnerability Remediation (Critical)

- **Upgrade Joomla immediately:** Upgrade Joomla to the latest stable version to patch the identified SQL injection vulnerability (Joomla! 3.2.x < 3.4.4).
- **Web Application Firewall (WAF):** Deploy a WAF rule to mitigate the specific SQL injection vulnerability before the upgrade can be completed.
- **Review Joomla Extensions:** Audit installed Joomla extensions for known vulnerabilities and update or remove any that are outdated or insecure.
- **Administrator Access:** Review administrator user accounts, enforce strong passwords and MFA where possible, and disable or remove any unused administrator accounts.

Exposed Directories (High)

- **Restrict Access to Sensitive Directories:** Implement web server configurations (e.g., .htaccess for Apache, or server configuration blocks for Nginx) to restrict access to sensitive directories like `/administrator`, `/installation`, `/tmp`, `/cache`, `/logs`, `/cli`, `/bin`, `/libraries`, `/language`, `/layouts`, `/plugins`, `/includes`, `/components`, `/modules`. Deny direct web access to these directories.
- **Remove Installation Directory:** If the `/installation` directory still exists, remove it immediately to prevent potential re-installation or configuration issues.

Account Compromise (High)

- **Password Reset:** Force a password reset for the compromised email address [redacted] and all associated accounts, including Joomla and any other accounts using that email address.

Figura 5.13: Generación de recomendaciones desde una perspectiva defensiva.

- **Interacción directa con la IA:** además de los botones predefinidos, el usuario puede realizar preguntas para resolver dudas específicas sobre el escaneo, como se muestra en la Figura 5.14.

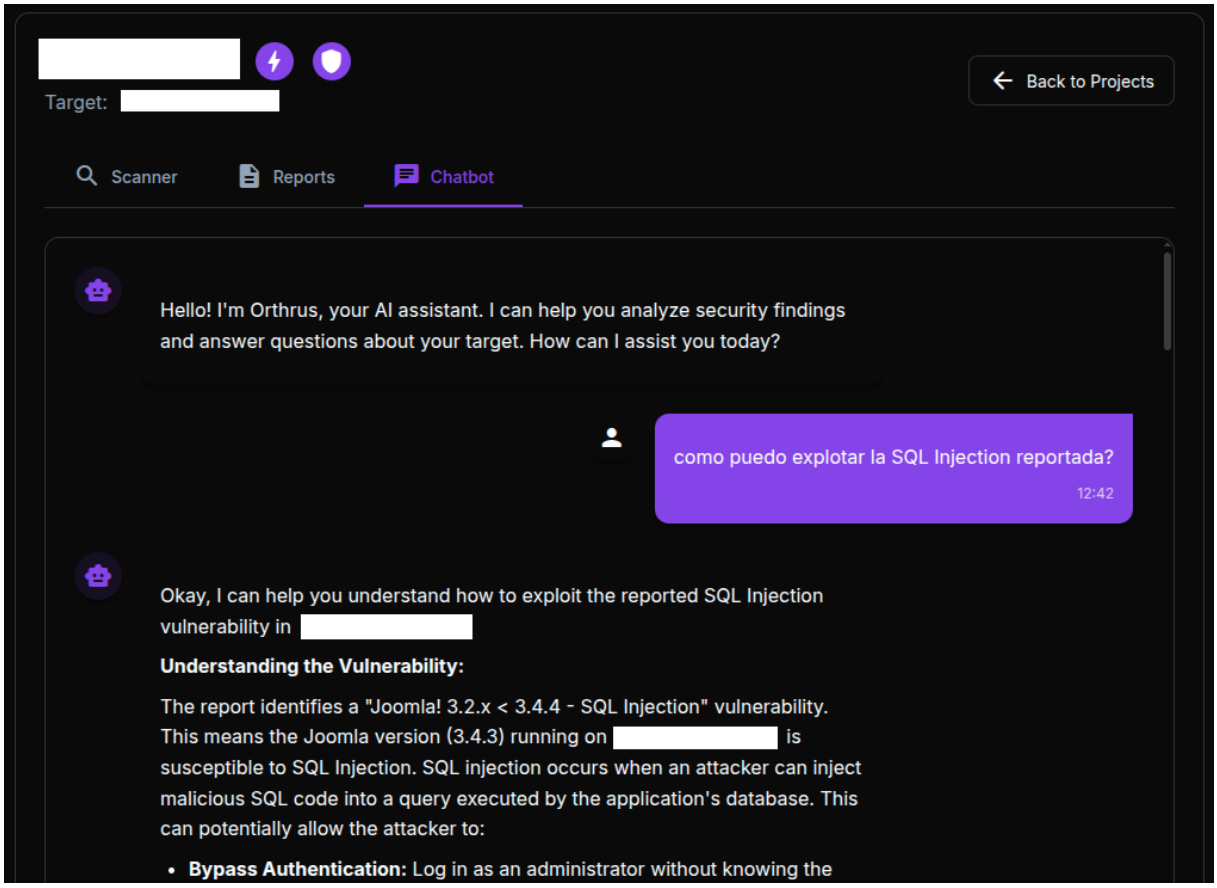


Figura 5.14: Interacción directa con la IA integrada en la herramienta.

Tras esto, el usuario cuenta con una base sólida de información obtenida durante la fase de reconocimiento, que le permite continuar con las siguientes etapas de la auditoría.

Capítulo 6

Resultados

Contenidos

6.1. Empresa A 77

6.1.1. Subdominio A1 79

6.1.2. Subdominio A2 82

6.1.3. Acciones recomendadas para la Empresa A 85

6.2. Empresa B 86

6.2.1. Acciones recomendadas para la Empresa B 90

6.3. Empresa C 92

6.4. Impacto en la eficiencia del análisis 93

6.4.1. Comparativa por fases 93

6.4.2. Resumen y análisis 98

En este capítulo se presentan los resultados obtenidos a partir de las pruebas realizadas con la herramienta desarrollada. Para ello, se han llevado a cabo una serie de escaneos sobre diferentes objetivos reales. Por cuestiones de confidencialidad, no se revelarán los nombres de las empresas ni los detalles específicos de los recursos analizados.

Se mostrará cómo la herramienta automatiza las distintas fases del reconocimiento, qué información es capaz de extraer, y cómo esta información puede aprovecharse para planificar acciones ofensivas o aplicar medidas defensivas.

Asimismo, se analizarán aspectos clave como el rendimiento, la calidad de los resultados, la interacción con el sistema de inteligencia artificial y la generación automática de informes.

6.1. Empresa A

La Empresa A se trata de una organización con actividad comercial a nivel nacional que dispone de una infraestructura informática amplia y distribuida. Sus sistemas incluyen varios servidores, redes segmentadas, múltiples servicios expuestos y un escaparate web bien estructurado. Estas características hacen que la superficie de ataque sea notablemente extensa.

Los resultados del escaneo realizado al dominio de la Empresa A se presentan resumidos en la Tabla 6.1.

Aspecto Analizado	Resultado
Puertos abiertos	5, incluyendo `SSH`, `FTP` y `HTTP`
URLs enumeradas	88
Proveedor de correo	IONOS
Proveedor de DNS	IONOS
Registro `DMARC`	No público, riesgo de *email spoofing*
CMS detectado	WordPress
Usuarios WordPress	2 usuarios enumerados
Plugins WordPress	5 plugins vulnerables
***Emails* comprometidos**	1 dirección comprometida
Subdominios enumerados	2 subdominios
Servicio FTP	Sin protección contra fuerza bruta

Tabla 6.1: Resumen del escaneo realizado a la Empresa A.

El escaneo se completó en un tiempo total de **23 minutos y 16 segundos**, como se puede observar en la Figura 6.1. Esta duración representa una mejora significativa en comparación con la ejecución manual de las mismas tareas, que requeriría considerablemente más tiempo y esfuerzo humano.

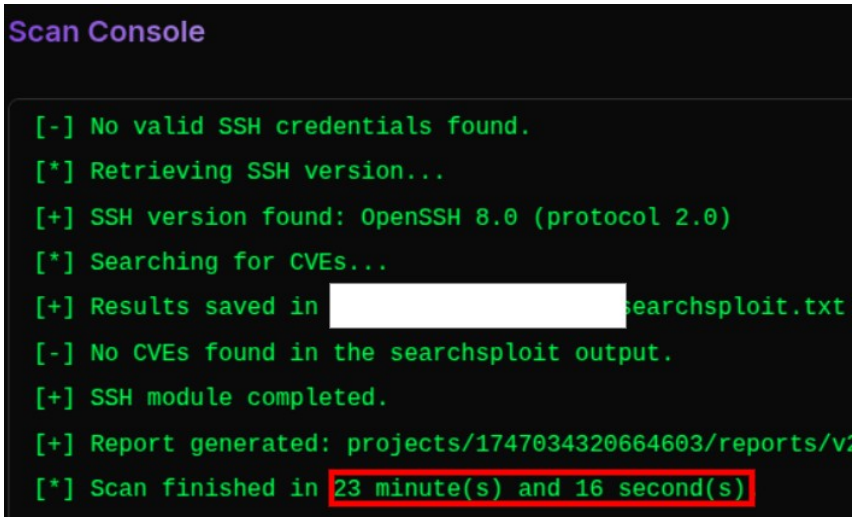


Figura 6.1: Duración del escaneo realizado a la Empresa A.

Entre las debilidades más destacadas se encuentra la **posibilidad de enumerar usuarios WordPress**, lo que expone al sistema a ataques de fuerza bruta y ataques dirigidos como *spear phishing* [61], especialmente si se combinan con la dirección de correo electrónico comprometida. Además, se podrían verificar manualmente los *CVEs* asociados a los plugins instalados para evaluar su impacto y aplicar parches si es necesario.

6.1.1. Subdominio A1

Una vez descubierto el subdominio **A1**, perteneciente a la Empresa A, se procedió a realizar un escaneo sobre él. La Tabla 6.2 resume los principales resultados obtenidos.

Aspecto Analizado	Resultado
Puertos abiertos	9, incluyendo puertos poco comunes como `8001` o `10050`
URLs enumeradas	41
Debilidades encontradas	1 vulnerabilidad web de criticidad media: **modo de depuración de Django habilitado**
Registro `DMARC`	No público, riesgo de *email spoofing*

Tabla 6.2: Resumen del escaneo realizado al subdominio A1.

El escaneo se llevó a cabo en **10 minutos y 57 segundos**, como se muestra en la Figura 6.2.

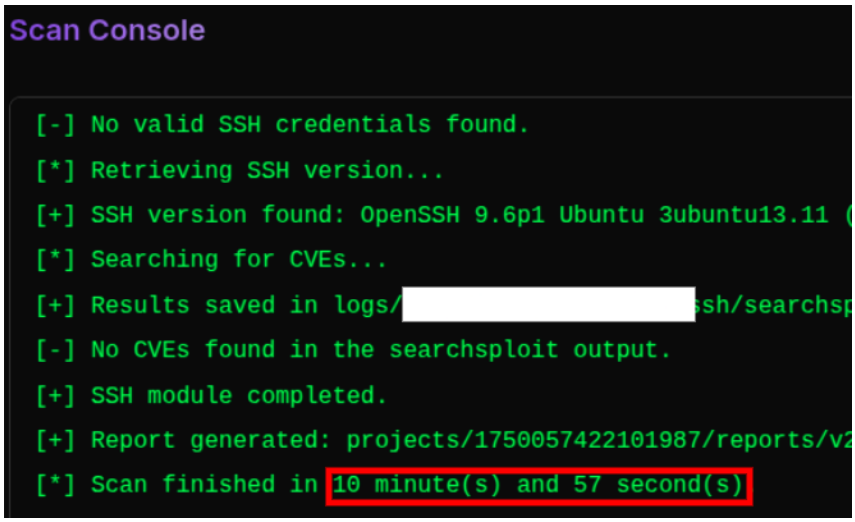


Figura 6.2: Duración del escaneo realizado al subdominio A1.

Entre los puertos abiertos destacan, además de los más habituales como `80` y `443`, otros menos comunes como `8001` y `10050`, los cuales podrían ser analizados más detalladamente de manera manual, ya que, al tratarse de puertos menos comunes, cabría la posibilidad de que estuviesen menos protegidos y presentasen brechas de seguridad. En la Figura 6.3 se muestran los puertos abiertos detectados en el subdominio A1.

Open services

Port	State	Service	Version
22/tcp	open	ssh	OpenSSH 9.6p1 Ubuntu 3ubuntu13.11 (Ubuntu Linux; protocol 2.0)
80/tcp	open	http	nginx 1.24.0 (Ubuntu)
443/tcp	open	ssl/http	nginx 1.24.0 (Ubuntu)
1720/tcp	open	h323q931?	
8000/tcp	open	http-alt	WSGIServer/0.2 CPython/3.10.17
8001/tcp	open	vcom-tunnel?	
10000/tcp	open	postgresql	PostgreSQL DB 9.6.0 or later
10001/tcp	open	nagios-nsca	Nagios NSCA
10050/tcp	open	tcpwrapped	

Figura 6.3: Puertos abiertos del subdominio A1.

Gracias al proceso de enumeración, se identificó una vulnerabilidad relacionada con el **modo de depuración de Django**, el cual se encontraba activo en un entorno que ya estaba en producción. Esta debilidad fue **verificada manualmente**, confirmando que permitía a un atacante obtener información sensible del servidor, como:

- Variables de entorno.
- Configuración de la base de datos.
- Fragmentos del código fuente de la aplicación.
- Más información sensible.

La Figura 6.4 muestra el mensaje reportado por el subdominio A1, indicando que el modo de depuración de Django estaba habilitado.

You're seeing this error because you have `DEBUG = True` in your Django settings file. Change that to `False`, and Django will display a standard 404 page.

Figura 6.4: Debilidad de Django encontrada en el subdominio A1.

Cabe destacar que, por razones de confidencialidad, los detalles sensibles no se muestran en este documento.

Adicionalmente, entre las rutas detectadas se encontró un endpoint asociado a una **API REST** accesible públicamente. La existencia simultánea de esta API junto con el modo de depuración activado puede amplificar el riesgo, ya que un atacante podría combinar ambos vectores para extraer información crítica del sistema o interferir con la lógica de la aplicación.

Si se combinan los puertos abiertos, la vulnerabilidad de Django y la API REST, se podría realizar un análisis exhaustivo de cada uno de los servicios expuestos, lo que podría llevar a la identificación de más debilidades o a la explotación de las existentes.

6.1.2. Subdominio A2

Una vez detectado el subdominio **A2**, también perteneciente a la Empresa A, se procedió a su análisis. Los resultados obtenidos se resumen en la Tabla 6.3.

Aspecto Analizado	**Resultado**
Puertos abiertos	11
URLs enumeradas	15
Debilidades encontradas	Modo `debug` de Django habilitado
Registro `DMARC`	No público, riesgo de *email spoofing*
Servicio FTP	Sin protección contra fuerza bruta

Tabla 6.3: Resumen del escaneo realizado al subdominio A2.

Este reconocimiento se completó en **12 minutos y 57 segundos**, como se muestra en la Figura 6.5.

```
Scan Console

[*] Obtaining the FTP version from the Nmap output...
[+] FTP version found: vsftpd 3.0.5
[*] Executing command: searchsploit "vsftpd 3.0.5" -j > lo
[+] Searchsploit result saved in logs[redacted]/f
[*] Searching for CVEs in the searchsploit output...
[-] No CVEs found in the searchsploit output.
[+] FTP module completed.
[+] Report generated: projects/1750058198958461/reports/v2
[*] Scan finished in 12 minute(s) and 57 second(s)
```

Figura 6.5: Duración del escaneo realizado al subdominio A2.

Al igual que en el subdominio A1, se identificaron servicios expuestos en puertos menos comunes, los cuales podrían ser analizados más detalladamente de forma manual. Éstos se muestran en la Figura 6.6.

Open services

Port	State	Service	Version
21/tcp	open	ftp	vsftpd 3.0.5
80/tcp	open	http	nginx 1.25.3
443/tcp	open	ssl/http	nginx 1.25.3
1720/tcp	open	h323q931?	
5432/tcp	open	postgresql	PostgreSQL DB 9.6.0 or later
8000/tcp	open	ssl/http	nginx 1.25.3
8080/tcp	open	http	(PHP 7.4.33)
8880/tcp	open	http	nginx 1.25.3
15432/tcp	open	postgresql	PostgreSQL DB 9.6.0 or later
20001/tcp	open	postgresql	PostgreSQL DB 9.6.0 or later
25432/tcp	open	postgresql	PostgreSQL DB 9.6.0 or later

Figura 6.6: Puertos abiertos del subdominio A2.

Asimismo, se detectó nuevamente la debilidad asociada al modo de depuración de Django activado. Esta vulnerabilidad permitió enumerar una gran cantidad de información interna del servidor. Entre los recursos más sensibles identificados se encuentra una **API REST completa**, con múltiples endpoints que permiten operaciones CRUD sobre diversos recursos. Además, la documentación de esta API se encontraba disponible públicamente, lo que facilita la prueba sistemática de cada endpoint en busca de posibles vulnerabilidades.

Otro punto crítico encontrado fue un servicio expuesto en el puerto `8080`, que corresponde a un panel de administración de **Adminer**, un sistema de gestión de bases de datos. Si se combina esta información con los datos obtenidos en el subdominio A1, como nombres de bases de datos y otras configuraciones, sería posible profundizar en la enumeración o incluso comprometer la base de datos. La Figura 6.7 muestra el panel de Adminer detectado en el subdominio A2.

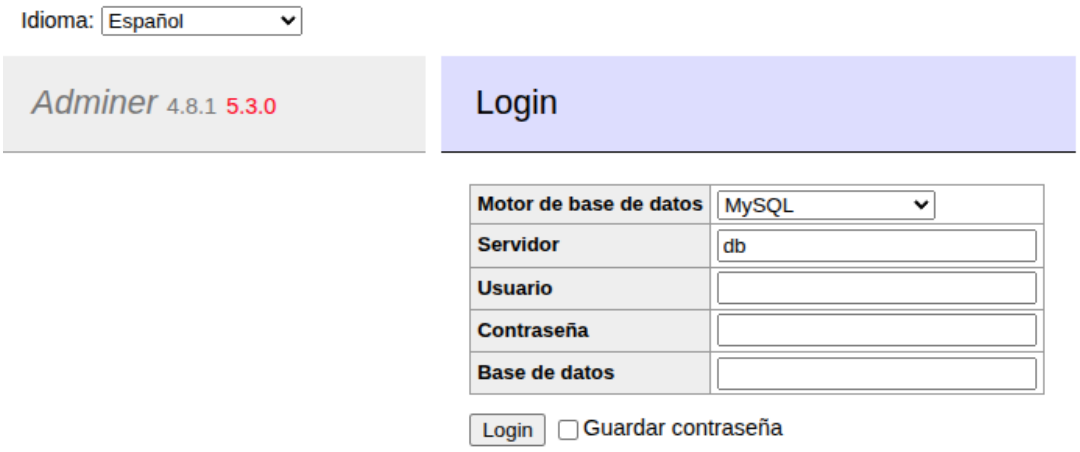


Figura 6.7: Panel de Adminer encontrado en el subdominio A2.

Cabe destacar que, si bien no se ha realizado una exploración completa de todos los vectores detectados, estos resultados ilustran con claridad el potencial de la herramienta para detectar debilidades y mapear la superficie de ataque. Este caso concreto demuestra hasta qué punto se puede avanzar en una auditoría únicamente con la información recopilada durante la fase de reconocimiento.

6.1.3. Acciones recomendadas para la Empresa A

Gracias a la información recopilada, la Empresa A podría considerar las siguientes medidas defensivas para mejorar la seguridad de sus sistemas:

- **Desactivar el modo de depuración de Django**, ya que expone información sensible del sistema. Esta medida debe considerarse prioritaria.
- **Revisar y auditar los servicios expuestos**, especialmente aquellos que utilizan puertos no estándar o poco comunes, ya que pueden presentar configuraciones por defecto o carecer de protección adecuada.
- **Evaluar la exposición de las APIs REST identificadas**. Si su exposición es necesaria, se deben aplicar medidas de seguridad como autenticación robusta, control de acceso por roles (RBAC) y validación de entrada.
- **Corregir la configuración de los registros** `DMARC` con el fin de evitar ataques de *email spoofing*.
- **Eliminar la posibilidad de enumerar usuarios en WordPress**, ya que esta información puede ser utilizada en ataques de fuerza bruta o *spear phishing*.
- **Verificar la vigencia de la dirección de correo electrónico comprometida**. En caso de seguir activa, se recomienda su reemplazo o, al menos, un cambio urgente de credenciales.
- **Asegurar el servicio FTP**, incorporando mecanismos como la limitación de intentos fallidos o bloqueo de direcciones IP sospechosas.

Además, la inteligencia artificial integrada en la herramienta puede ser utilizada para generar recomendaciones sobre las medidas a tomar. En la Figura 6.8 se muestran algunas de las sugerencias defensivas propuestas por la IA para el subdominio A1.

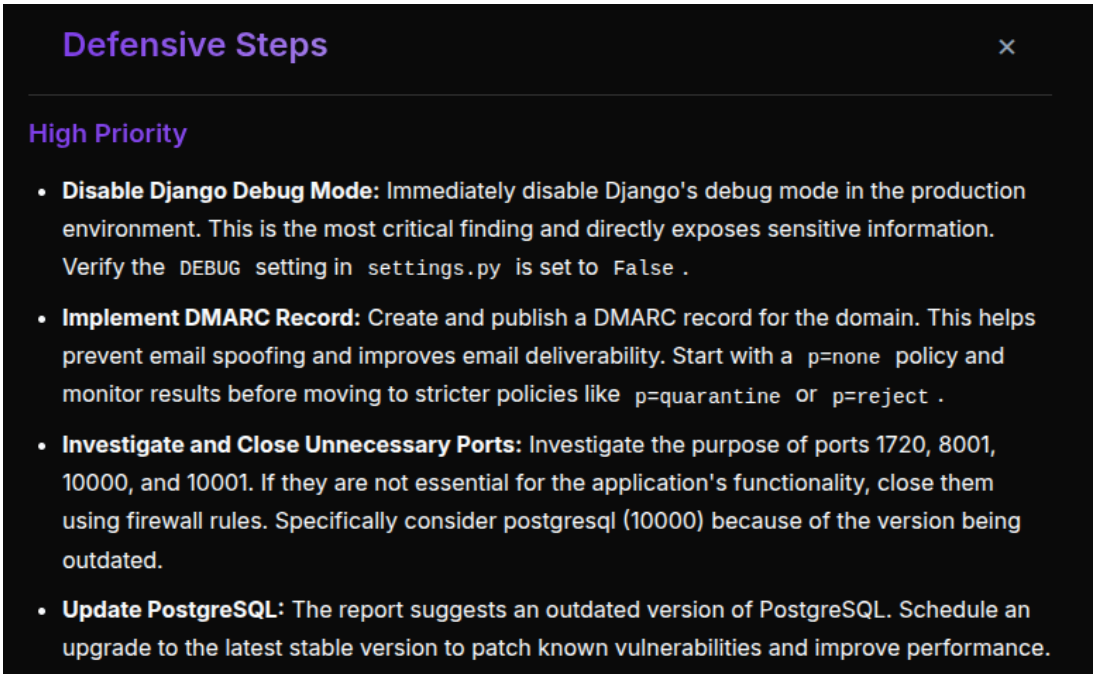


Figura 6.8: Sugerencias defensivas propuestas por la IA para el subdominio A1.

6.2. Empresa B

La Empresa B se trata de una propuesta no comercial, cuyo único propósito es ofrecer información sin base económica. El sitio web principal actúa únicamente como escaparate informativo con temática deportiva. Su infraestructura informática es reducida, sin aplicaciones complejas.

Los resultados del escaneo realizado al dominio de la Empresa B se presentan resumidos en la Tabla 6.4.

Elemento	Resultado
Puertos abiertos	4
URLs enumeradas	56
Proveedor de correo	ARUBA-ASN, IT
Proveedor de DNS	ARUBA-ASN, IT e INTERNET-CZ Ktis, CZ
CMS detectado	Joomla, versión 3.4.3
Vulnerabilidades encontradas	Vulnerabilidad **SQL Injection** de la versión de Joomla
***Emails* comprometidos**	1 dirección comprometida
Subdominios enumerados	1 subdominio

Tabla 6.4: Resumen del escaneo realizado a la Empresa B.

El escaneo se completó en un tiempo total de **13 minutos y 9 segundos**, como se muestra en la Figura 6.9.

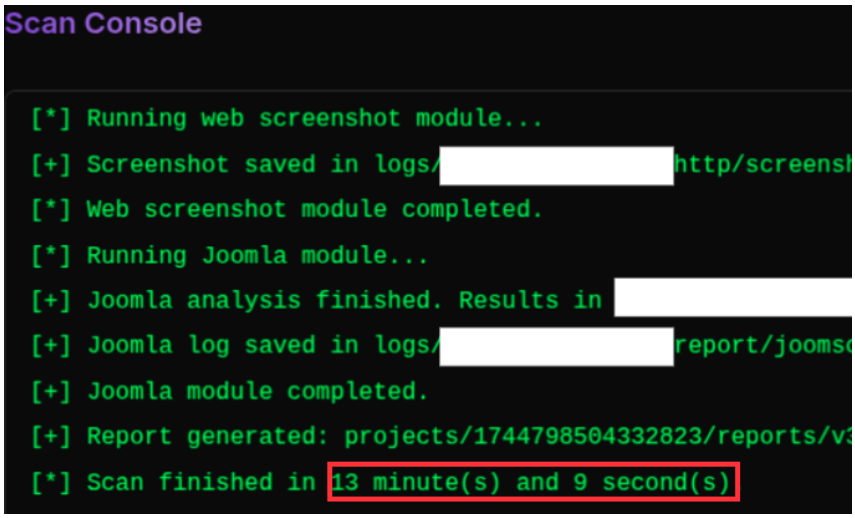


Figura 6.9: Duración del escaneo realizado a la Empresa B.

Entre los hallazgos más relevantes destaca la detección del servicio `SSH` en el puerto 2222 en lugar del puerto estándar 22. Este hecho puede indicar que se trata de una

instancia en fase de pruebas, lo que puede aumentar el riesgo de una posible explotación. La Figura 6.10 muestra los puertos abiertos detectados en la Empresa B.

Open services

Port	State	Service	Version
80/tcp	open	http	aruba-proxy
443/tcp	open	ssl/https	aruba-proxy
1720/tcp	open	h323q931?	
2222/tcp	open	ssh	OpenSSH 9.6p1 Ubuntu 3ubuntu13.11 (Ubuntu Linux; protocol 2.0)

Figura 6.10: Puertos abiertos de la Empresa B.

Por otro lado, la herramienta ha detectado que el CMS utilizado es **Joomla**, en su versión **3.4.3**, considerablemente desactualizada respecto a la versión actual (5.3.1 en el momento del análisis). Esta versión incluye una vulnerabilidad crítica de tipo *SQL Injection*. En el informe generado automáticamente por la herramienta se proporciona el enlace directo a un repositorio público que contiene un exploit para esta vulnerabilidad, como se muestra en la Figura 6.11.

Core Vulnerability

Joomla! 3.2.x < 3.4.4 - SQL Injection
EDB : https://www.exploit-db.com/exploits/38534/

Figura 6.11: Vulnerabilidad SQL Injection de Joomla detectada.

Además, se ha identificado el panel de autenticación de Joomla expuesto, lo que abre la posibilidad de lanzar ataques de fuerza bruta o intentar explotar la vulnerabilidad detectada directamente desde el formulario de login. La Figura 6.12 muestra el panel de acceso de Joomla detectado en la Empresa B.

Figura 6.12: Panel de acceso de Joomla detectado en la Empresa B.

Adicionalmente, se ha detectado una dirección de correo electrónico comprometida. Esta información, combinada con el resto de hallazgos, podría facilitar la ejecución de ataques de tipo *spear phishing* dirigidos a usuarios específicos, o incluso ser utilizada para realizar ataques de fuerza bruta contra el panel de acceso de Joomla.

La herramienta también permite interactuar con la inteligencia artificial para generar recomendaciones sobre acciones ofensivas a realizar. En la Figura 6.13 se muestran algunas de las sugerencias propuestas por la IA para continuar la auditoría de seguridad en la Empresa B.

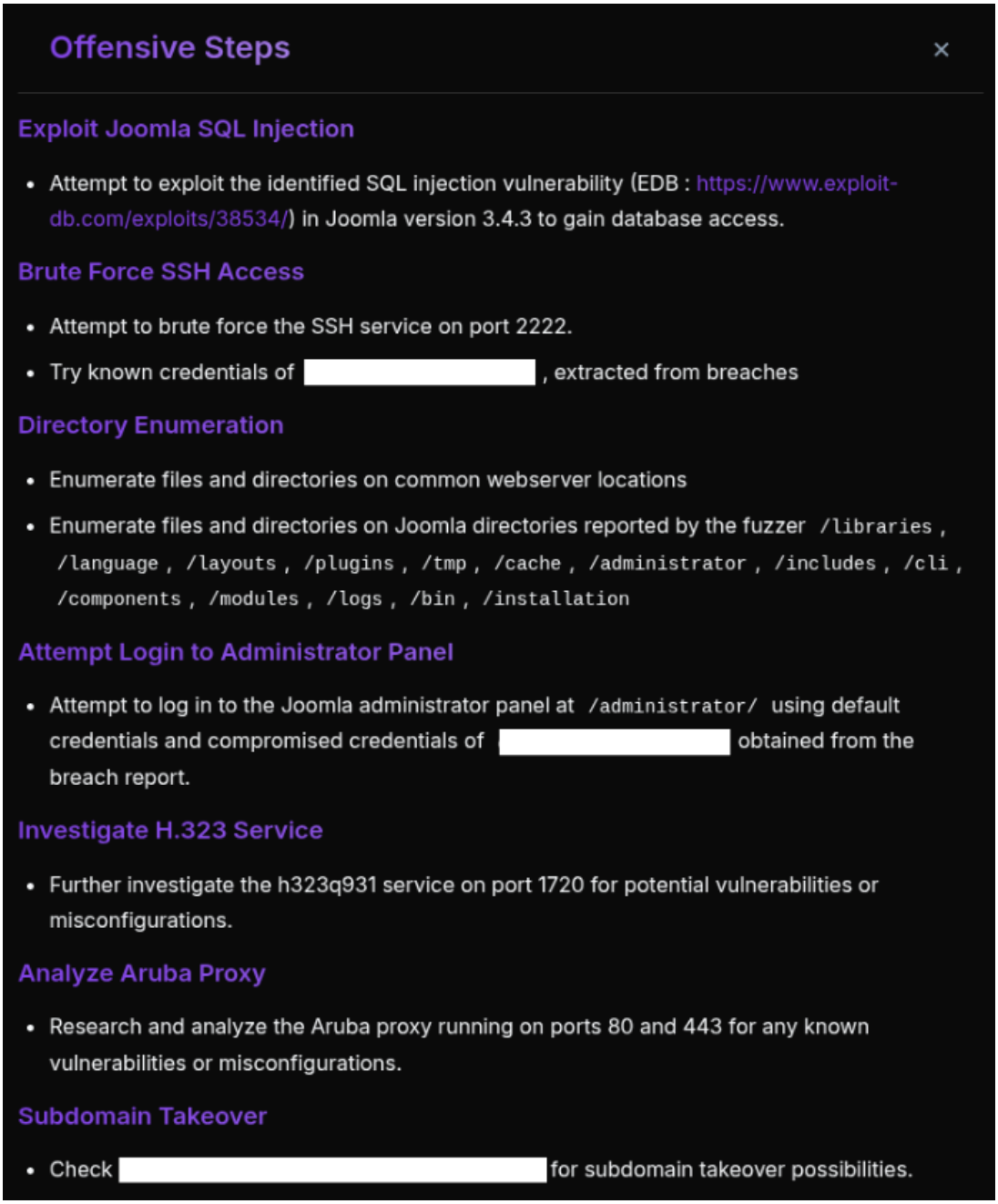


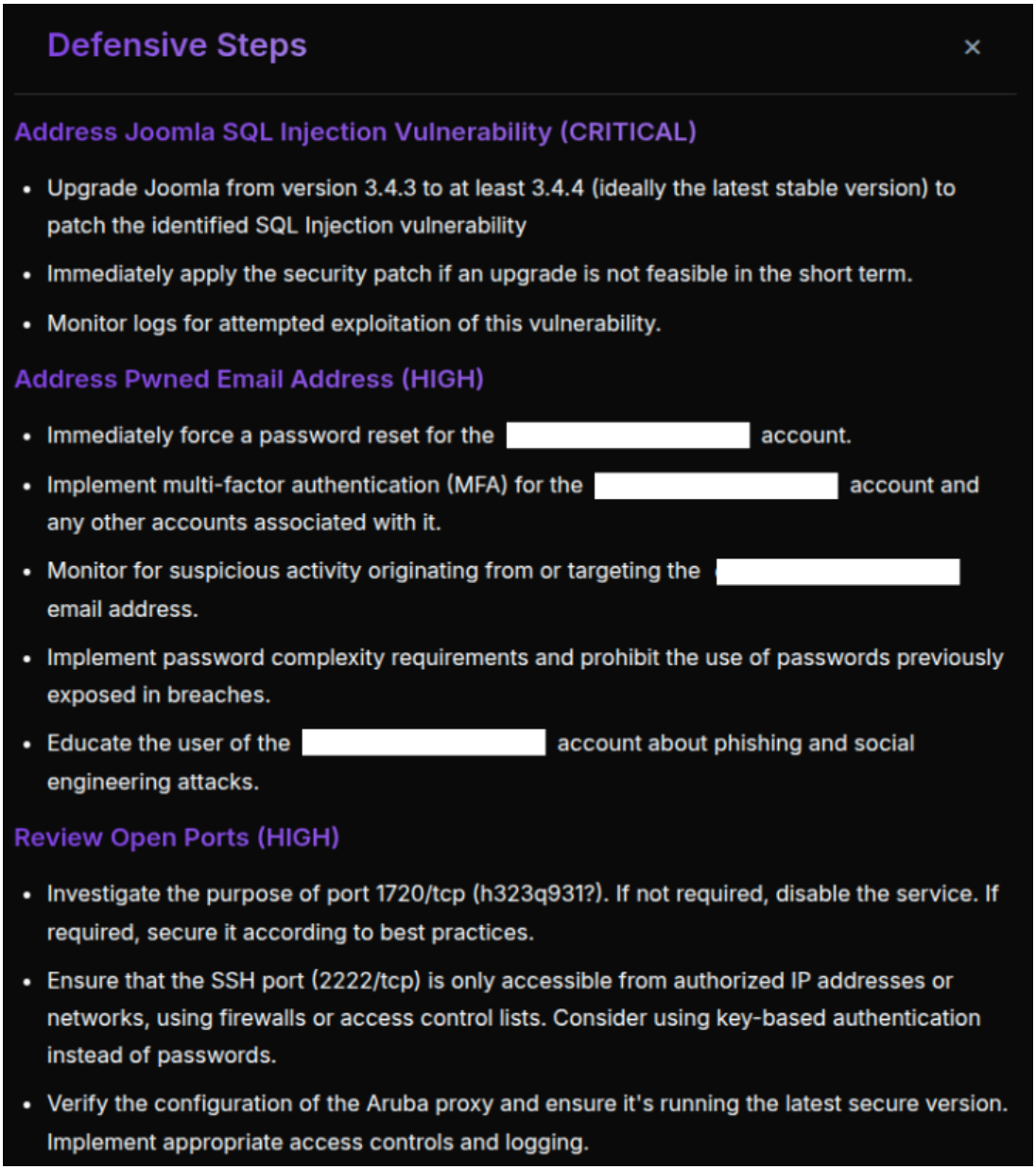
Figura 6.13: Sugerencias ofensivas propuestas por la IA para la Empresa B.

6.2.1. Acciones recomendadas para la Empresa B

A partir de la información obtenida durante el escaneo, la Empresa B podría considerar las siguientes acciones defensivas:

- **Actualizar Joomla a su versión más reciente**, con el fin de corregir la vulnerabilidad crítica de *SQL Injection* presente en la versión actualmente instalada (3.4.3).
- **Revisar la dirección de correo electrónico comprometida**: verificar si sigue en uso y, en caso afirmativo, proceder a su sustitución y cambio de credenciales asociadas.
- **Analizar la configuración de los servicios expuestos**, especialmente el servicio SSH presente en el puerto 2222, para comprobar si su exposición es intencionada y se encuentra adecuadamente protegido.
- **Evaluar la necesidad del subdominio enumerado**. Si no es necesario, se recomienda eliminarlo para reducir la superficie de ataque.

Estas acciones también pueden ser generadas por la inteligencia artificial integrada. En la Figura 6.14 se muestran las sugerencias defensivas propuestas por la IA para la Empresa B.



Defensive Steps

Address Joomla SQL Injection Vulnerability (CRITICAL)

- Upgrade Joomla from version 3.4.3 to at least 3.4.4 (ideally the latest stable version) to patch the identified SQL Injection vulnerability
- Immediately apply the security patch if an upgrade is not feasible in the short term.
- Monitor logs for attempted exploitation of this vulnerability.

Address Pwned Email Address (HIGH)

- Immediately force a password reset for the [redacted] account.
- Implement multi-factor authentication (MFA) for the [redacted] account and any other accounts associated with it.
- Monitor for suspicious activity originating from or targeting the [redacted] email address.
- Implement password complexity requirements and prohibit the use of passwords previously exposed in breaches.
- Educate the user of the [redacted] account about phishing and social engineering attacks.

Review Open Ports (HIGH)

- Investigate the purpose of port 1720/tcp (h323q931?). If not required, disable the service. If required, secure it according to best practices.
- Ensure that the SSH port (2222/tcp) is only accessible from authorized IP addresses or networks, using firewalls or access control lists. Consider using key-based authentication instead of passwords.
- Verify the configuration of the Aruba proxy and ensure it's running the latest secure version. Implement appropriate access controls and logging.

Figura 6.14: Sugerencias defensivas propuestas por la IA para la Empresa B.

6.3. Empresa C

Empresa C

6.4. Impacto en la eficiencia del análisis

Con el objetivo de evaluar el impacto real de la herramienta desarrollada en la eficiencia del análisis, se ha realizado una comparación. Para ello, se ha replicado el escaneo realizado a la Empresa B de manera completamente manual, empleando las mismas herramientas utilizadas por la herramienta, pero sin el soporte de esta.

Este análisis se ha llevado a cabo desde la perspectiva de un auditor que no memoriza todos los comandos ni sus parámetros específicos, por lo que se incluye también el tiempo necesario para consultar documentación, manuales, recursos personales o realizar búsquedas en Internet. Esta situación representa un caso más realista para muchos profesionales del sector.

6.4.1. Comparativa por fases

A continuación, se detallan las fases del reconocimiento realizadas manualmente, junto con el tiempo invertido en cada una de ellas.

- **Planificación inicial:** el auditor debe elaborar una hoja de ruta con las tareas a ejecutar. Este paso consume aproximadamente **10 minutos**, mientras que la herramienta ya cuenta con una ejecución automatizada predefinida.
- **Escaneo de puertos y servicios:** se realizan dos escaneos con `Nmap`: el primero para obtener los puertos abiertos y el segundo para enumerar los servicios y versiones que se ejecutan en esos puertos. Entre la consulta de parámetros (**5 minutos** para cada escaneo), la ejecución del escaneo básico (**26 segundos**) y detallado (**2 minutos y 25 segundos**), y el registro de los resultados para el posterior informe (**3 minutos**), el tiempo total asciende a **15 minutos**.

```
gordillo7@gordillo7-Z790-PG-Riptide:~$ time nmap -p- --open --min-rate 5000 -n -
Pn [redacted]
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-06-25 09:30 CEST
Nmap scan report for [redacted]
Host is up (0.0023s latency).
Not shown: 65531 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT     STATE SERVICE
80/tcp   open  http
443/tcp  open  https
1720/tcp open  h323q931
2222/tcp open  EtherNetIP-1

Nmap done: 1 IP address (1 host up) scanned in 26.45 seconds

real    0m26,457s
user    0m10,048s
```

Figura 6.15: Tiempo empleado en el escaneo inicial.

```
Service detection performed. Please report any incorrect results at https://
nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 145.83 seconds

real    2m25,838s
user    0m0,194s
sys     0m0,040s
gordillo7@gordillo7-Z790-PG-Riptide:~$
```

Figura 6.16: Tiempo empleado en el escaneo detallado.

- **Reconocimiento DNS:** esta fase se divide en tres tareas principales. En primer lugar, se consultan los registros del dominio utilizando `DNSDumpster`, lo cual, sumando el tiempo de revisión de los resultados y su documentación para el informe, lleva aproximadamente **5 minutos**. A continuación, se utiliza la herramienta `MXToolBox` para verificar el estado del registro `DMARC`, lo cual requiere en torno a **1 minuto**. Por último, se realiza una consulta `WHOIS` mediante la misma API utilizada por la herramienta desarrollada, lo que añade unos **2 minutos** más. El tiempo total invertido en esta fase asciende a **8 minutos**.

- **OSINT sobre correos:** en esta fase se utilizan dos herramientas. Primero, se emplea `IntelligenceX` para localizar direcciones de correo asociadas al dominio objetivo, tarea que, incluyendo el registro de los resultados para el informe, requiere

unos **3 minutos**. Posteriormente, se consultan estas direcciones en `LeakCheck` para verificar si alguna ha sido comprometida en brechas de datos, lo cual añade aproximadamente **2 minutos** adicionales. El tiempo total dedicado a esta fase es de **5 minutos**.

- **Enumeración de subdominios:** se utiliza la herramienta `Subfinder` para identificar subdominios asociados al dominio principal. La consulta de los parámetros necesarios y la ejecución del comando consume aproximadamente **1 minuto**, siendo la ejecución en sí muy rápida (tan solo **2 segundos**, como se muestra en la Figura 6.17). A esto se suma el tiempo destinado a documentar los subdominios encontrados, **1 minuto** adicional. El tiempo total invertido en esta fase asciende a **2 minutos**.

```
gordillo7@gordillo7-Z790-PG-Riptide:~$ time subfinder -d [redacted]
-silent
[redacted]

real    0m2,290s
user    0m0,105s
sys     0m0,038s
```

Figura 6.17: Tiempo de ejecución de Subfinder.

- **Escaneo de vulnerabilidades web:** se utiliza la herramienta `Nuclei` para detectar vulnerabilidades en servicios web del objetivo. Se han invertido **5 minutos** en revisar la documentación de la herramienta. El escaneo ha tardado **1 minuto y 4 segundos** en completarse, como se puede ver en la Figura 6.18. En este caso no se detectaron vulnerabilidades, por lo que no fue necesario registrar los resultados. El tiempo total empleado en esta fase asciende a **6 minutos**.

```
[INF] Current nuclei version: v3.4.2 (outdated)
[INF] Current nuclei-templates version: v10.2.3 (latest)
[WRN] Scan results upload to cloud is disabled.
[INF] New templates added in latest release: 105
[INF] Templates loaded for current scan: 5298
[INF] Executing 5283 signed templates from projectdiscovery/nuclei-templa
tes
[WRN] Loading 15 unsigned templates for scan. Use with caution.
[INF] Targets loaded for current scan: 1
[INF] Running httpx on input host
[INF] Found 1 URL from httpx
[INF] Templates clustered: 394 (Reduced 342 Requests)
[INF] Using Interactsh Server: oast.site
[INF] No results found. Better luck next time!

real    1m4,097s
user    0m9,096s
sys     0m1,795s
```

Figura 6.18: Tiempo de ejecución de Nuclei.

- **Fuzzing web:** para la enumeración de rutas y recursos web se emplea la herramienta `Feroxbuster`. Dado que cuenta con múltiples parámetros y opciones de configuración, se han invertido aproximadamente **10 minutos** en revisar su documentación y preparar el comando adecuado. La ejecución del escaneo ha durado **38 segundos**, como se muestra en la Figura 6.19. Posteriormente, se han dedicado **2 minutos** a registrar los resultados obtenidos. El tiempo total destinado a esta fase asciende a **12 minutos**.

```
real    0m38,062s
user    0m2,348s
sys     0m0,914s
```

Figura 6.19: Tiempo de ejecución de Feroxbuster.

- **Detección de CMS:** para identificar el gestor de contenidos utilizado en el sitio, se emplea `WhatWeb`. La consulta de su documentación ha requerido aproximadamente **1 minuto**. La ejecución del comando ha tardado **3 segundos**, como se muestra en la Figura 6.20, y el registro de los resultados se ha realizado en aproximadamente **1 minuto**. En conjunto, esta fase ha supuesto un tiempo total de **2 minutos**.

```
gordillo7@gordillo7-Z790-PG-Riptide:~$ time whatweb [redacted]
[redacted] [301 Moved Permanently] Country[ROMANIA][RO], HTTPS
erver[aruba-proxy], [redacted], RedirectLocation[redacted]
[redacted], Title[301 Moved Permanently], UncommonHeaders[x-servername,alt-s
vc]
[redacted] [200 OK] Cookies[cefc1d3521cc03d09ee61bfb76eb7
406], Country[ROMANIA][RO], HTML5, HTTPServer[aruba-proxy], HttpOnly[cefc1d
3521cc03d09ee61bfb76eb7406], [redacted] MetaGenerator[Joomla! - Open
 Source Content Management], Script[application/javascript,text/javascript]
, Strict-Transport-Security[max-age=31536000; includeSubDomains; preload],
TheHostingTool[1,11,2,4,5,7], [redacted]
[redacted], UncommonHeaders[x-servername,alt-svc], X-UA-Compatible[IE=edge]

real    0m3,658s
user    0m3,618s
sys     0m0,040s
```

Figura 6.20: Tiempo de ejecución de WhatWeb.

- **Reconocimiento Joomla:** tras detectar que el CMS utilizado era `Joomla`,

se empleó la herramienta `JoomScan` para identificar posibles vulnerabilidades específicas. La consulta de su documentación y opciones disponibles requirió unos **5 minutos**. La ejecución del escaneo tomó **2 minutos y 43 segundos**, como se puede observar en la Figura 6.21, y el registro de los resultados para el informe supuso aproximadamente **10 minutos**. En total, esta fase requirió unos **17 minutos**.

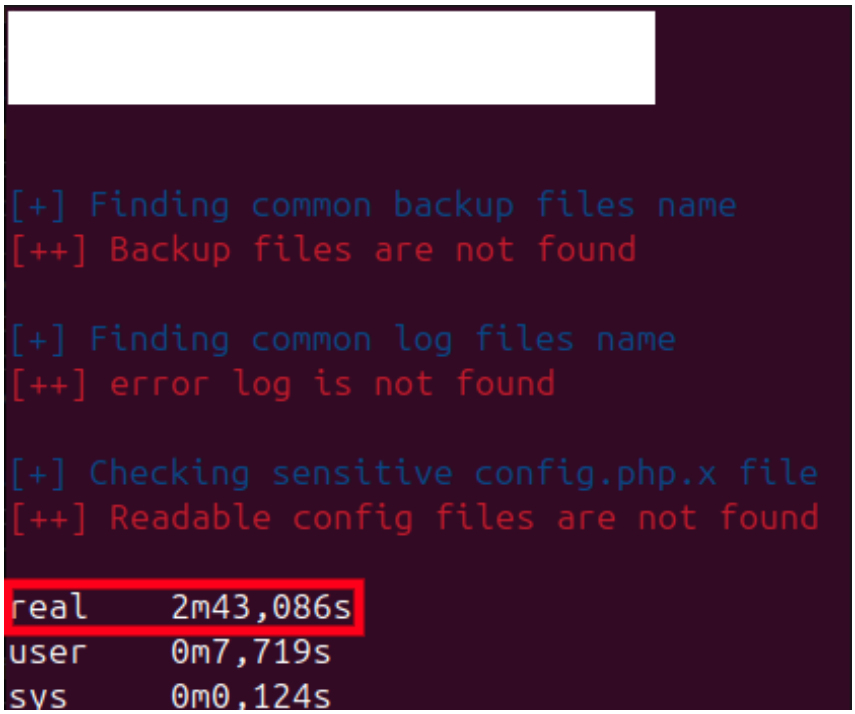


Figura 6.21: Tiempo de ejecución de JoomScan.

- **Reconocimiento FTP / SSH:** en este caso no se detectó el puerto 21 para FTP ni el 22 para SSH, por lo que no se realizaron acciones para esta fase.
- **Redacción del informe:** la redacción manual del informe técnico ha requerido aproximadamente **20 minutos**.

6.4.2. Resumen y análisis

En total, el proceso manual de reconocimiento ha supuesto un tiempo estimado de **97 minutos**, sin contar posibles errores humanos o distracciones. En comparación, la

herramienta completó todo este proceso en **13 minutos y 9 segundos**, como se mostró anteriormente en la Figura 6.9, lo que supone una mejora considerable en términos de eficiencia, reduciendo el tiempo invertido en más de **80 minutos**.

Además del ahorro de tiempo, cabe destacar que:

- La herramienta automatiza tareas repetitivas, elimina la necesidad de consultar documentación técnica constantemente y garantiza que no se omitan fases del proceso.
- La integración de inteligencia artificial permite obtener sugerencias de medidas defensivas u ofensivas de manera inmediata, sin necesidad de razonamiento excesivo por parte del auditor. Esto agiliza la transición a fases posteriores de la auditoría, como la explotación.
- Cuantos más servicios incorpore el objetivo, mayor será el impacto positivo de la herramienta.

Por todo ello, puede concluirse que el uso de la herramienta propuesta no solo acelera de forma notable el proceso de reconocimiento, sino que además mejora su calidad y consistencia, permitiendo que el auditor se centre en las acciones que realmente requieren intervención humana.

Capítulo 7

Conclusiones y trabajos futuros

Contenidos

7.1. Conclusiones **101**

7.2. Líneas de trabajo futuras **102**

7.1. Conclusiones

El presente Trabajo de Fin de Grado tenía como objetivo principal el desarrollo de una herramienta capaz de automatizar la fase de reconocimiento en auditorías de seguridad, generando informes técnicos estructurados de manera automática y permitiendo la integración de inteligencia artificial para asistir en el análisis de resultados. Tras el desarrollo e implementación de la herramienta, se puede afirmar que se han cumplido satisfactoriamente los siguientes objetivos:

- Se ha desarrollado una herramienta que automatiza las tareas fundamentales de reconocimiento, desde el escaneo de puertos y servicios hasta la enumeración de subdominios, detección de CMS, escaneo de vulnerabilidades y análisis de servicios como FTP o SSH.
- Se ha implementado un sistema de generación de informes en formato PDF que organiza de manera clara y estructurada los resultados obtenidos durante el escaneo.
- Se ha integrado un asistente de inteligencia artificial basado en modelos generativos, que permite al usuario consultar recomendaciones, resolver dudas y planificar los siguientes pasos de la auditoría.
- Se ha desarrollado una interfaz gráfica sencilla e intuitiva, que permite la gestión de proyectos, configuración de opciones, visualización de resultados y versionado de informes.
- Se ha ofrecido también la posibilidad de ejecución por consola, aportando versatilidad al uso de la herramienta para perfiles más técnicos.
- Se ha diseñado una arquitectura modular y escalable, que facilita la incorporación de nuevas funcionalidades en el futuro.

Los resultados presentados en el **Capítulo 6** confirman la utilidad y eficacia de la herramienta en escenarios reales. El tiempo de análisis se reduce considerablemente

respecto a un enfoque manual, y la calidad de los datos recopilados proporciona una base sólida para fases posteriores del proceso de auditoría.

7.2. Líneas de trabajo futuras

Durante el desarrollo del proyecto han surgido nuevas ideas y oportunidades de mejora las cuales no han podido ser implementadas por falta de tiempo, por lo que se proponen como líneas de trabajo futuras:

- **Mejora de la interfaz**: se plantea la migración de la interfaz actual basada en HTML, CSS y JavaScript a un framework como React o Next.js, con el fin de mejorar la organización del código, facilitar el mantenimiento y ofrecer una experiencia de usuario más agradable.
- **Personalización de informes**: una mejora destacada sería permitir la edición de los informes generados, permitiendo al auditor añadir notas personalizadas, observaciones manuales o incluir fases adicionales como explotación o post-explotación.
- **Mejoras en la presentación del informe**: rediseñar el formato visual del PDF generado para hacerlo más claro, atractivo y profesional, incluyendo tablas comparativas, gráficos o enlaces interactivos.
- **Ampliación de servicios analizados**: incluir nuevas fases del proceso de reconocimiento, así como la posibilidad de realizar algunas tareas pertenecientes a otras fases de una auditoría, como la explotación.
- **Programación de escaneos automáticos**: una funcionalidad que permita programar escaneos automáticos en un horario definido por el usuario, con el objetivo de realizar análisis periódicos sin intervención manual.

Estas mejoras permitirían convertir la herramienta en una solución aún más completa, capaz de abordar nuevas fases del proceso de auditoría y adaptarse a entornos corporativos más exigentes.

Bibliografía

[1] Carlos Cilleruelo. Fases de un pentest, 2024. Artículo sobre las fases de una auditoría de seguridad informática. URL: https://keepcoding.io/blog/fases-de-un-pentest-ciberseguridad/.

[2] MITRE ATTACK. Reconnaissance, 2020. Fase de reconocimiento en auditorías de seguridad. URL: https://attack.mitre.org/tactics/TA0043/.

[3] James P. Anderson. Computer security technology planning study, volume 1. Technical report, U.S. Air Force Electronic Systems Division, 1972. Informe pionero sobre planificación y análisis de seguridad informática, volumen 1. URL: https://csrc.nist.gov/files/pubs/conference/1998/10/08/proceedings-of-the-21st-nissc-1998/final/docs/early-cs-papers/ande72a.pdf.

[4] James P. Anderson. Computer security technology planning study, volume 2. Technical report, U.S. Air Force Electronic Systems Division, 1972. Informe pionero sobre planificación y análisis de seguridad informática, volumen 2. URL: https://csrc.nist.rip/publications/history/ande72.pdf.

[5] Dan Farmer and Wietse Venema. Satan: Security administrator tool for analyzing networks, 1995. Herramienta pionera de escaneo de vulnerabilidades en redes UNIX. URL: http://www.porcupine.org/satan.

[6] ISECOM. Isecom - institute for security and open methodologies. Sitio web oficial de ISECOM. URL: https://www.isecom.org.

[7] OWASP Foundation. Open web application security project (owasp), 2001. Sitio web oficial de OWASP. URL: https://owasp.org.

[8] ProjectDiscovery. Projectdiscovery cloud platform, 2024. Plataforma de escaneo automático basada en herramientas open source. URL: https://cloud.projectdiscovery.io/.

[9] Intruder Systems Ltd. Intruder.io - automated vulnerability scanner, 2024. Servicio de escaneo de vulnerabilidades orientado a empresas. URL: https://intruder.io/.

[10] Detectify AB. Detectify - web vulnerability scanner powered by ethical hackers, 2024. Plataforma de escaneo web basada en inteligencia colectiva. URL: https://detectify.com/.

[11] INCIBE. ¿qué es una vulnerabilidad zero day?, 2020. Publicación de INCIBE sobre vulnerabilidades de día cero. URL: https://www.incibe.es/ciudadania/blog/que-es-una-vulnerabilidad-zero-day.

[12] OWASP Foundation. *OWASP Web Security Testing Guide v4.2*, 2020. Metodología en el marco de OWASP. URL: https://owasp.org/www-project-web-security-testing-guide/v42/.

[13] MITRE Corporation. *MITRE ATTACK*, 2015. Metodología en el marco de MITRE ATTACK. URL: https://attack.mitre.org/.

[14] Pete Herzog. *OSSTMM – Open Source Security Testing Methodology Manual*, 2001. Metodología en el marco de OSSTMM. URL: https://www.isecom.org/OSSTMM.3.pdf.

[15] Nmap Project. Nmap - network mapper, 1997. Sitio web oficial de Nmap, herramienta de escaneo de puertos y servicios. URL: https://nmap.org.

[16] Robert Graham. Masscan: Mass ip port scanner. Repositorio oficial en GitHub del escáner de puertos Masscan. URL: https://github.com/robertdavidgraham/masscan.

[17] Bee-San. Rustscan: The modern port scanner. Repositorio oficial en GitHub del escáner de puertos RustScan. URL: https://github.com/bee-san/RustScan.

[18] HackerTarget.com. Dnsdumpster - online dns reconnaissance tool. Herramienta para visualización de registros DNS. URL: https://dnsdumpster.com/.

[19] Inc. MXToolbox. Mxtoolbox. Herramienta para análisis de registros DNS. URL: https://mxtoolbox.com/.

[20] Wikipedia contributors. Email spoofing. Definición del término .Email spoofing". URL: https://es.wikipedia.org/wiki/Email_spoofing.

[21] API Ninjas. Whois api - api ninjas. API REST para realizar consultas WHOIS sobre nombres de dominio. URL: https://www.api-ninjas.com/api/whois.

[22] Carlos Perez. dnsrecon - dns enumeration script. Script para enumeración y análisis de registros DNS. URL: https://github.com/darkoperator/dnsrecon.

[23] Filip Waeytens. dnsenum - multithreaded perl script to enumerate dns information. Script para descubrimiento de información DNS. URL: https://github.com/fwaeytens/dnsenum.

[24] OWASP Foundation. Amass - in-depth dns enumeration and attack surface mapping. Herramienta para mapeo de superficie de ataque. URL: https://github.com/owasp-amass/amass.

[25] EasyDMARC. Easydmarc – your dmarc journey made simple. Plataforma de validación de registro DMARC. URL: https://easydmarc.com/.

[26] Who.is. Domain name information lookup. Herramienta para realizar consultas WHOIS de dominios. URL: https://who.is/.

[27] WhoisXML API. Whoisxml api. API para realizar consultas WHOIS. URL: https://es.whoisxmlapi.com/.

[28] IntelligenceX.io. Intelligencex. Buscador OSINT para datos filtrados, archivos expuestos y metadatos de dominios. URL: https://intelx.io/.

[29] LeakCheck.io. Leakcheck - find out if your credentials have been compromised. Web de consulta de correos electrónicos comprometidos en filtraciones públicas. URL: https://leakcheck.io/.

[30] Have I Been Pwned? Have i been pwned? - check if your email has been compromised in a data breach. Web de consulta de correos electrónicos comprometidos en filtraciones públicas. URL: https://haveibeenpwned.com/.

[31] BreachDirectory.org. Breachdirectory - check if your email or username was compromised. Web de consulta de correos electrónicos comprometidos en filtraciones públicas. URL: https://breachdirectory.org/.

[32] ProjectDiscovery. Subfinder - subdomain discovery tool. Herramienta de enumeración de subdominios a partir de fuentes públicas. URL: https://github.com/projectdiscovery/subfinder.

[33] ProjectDiscovery. Httpx - fast and multi-purpose http toolkit. Herramienta para validación de hosts vía HTTP. URL: https://github.com/projectdiscovery/httpx.

[34] Sectigo Limited. crt.sh - certificate search. Buscador de certificados SSL/TLS para identificación de subdominios. URL: https://crt.sh/.

[35] Carlos Cilleruelo. ¿qué es red team en ciberseguridad?, 2024. Artículo sobre el rol del Red Team en el ámbito de la ciberseguridad. URL: https://keepcoding.io/blog/que-es-red-team-en-ciberseguridad/.

[36] ProjectDiscovery. Nuclei. Herramienta de escaneo de vulnerabilidades web. URL: https://github.com/projectdiscovery/nuclei.

[37] Sullo. Nikto - web server scanner. Herramienta de escaneo de vulnerabilidades web. URL: https://github.com/sullo/nikto.

[38] Andrew Horton (urbanadventurer). Whatweb - next generation web scanner. Herramienta para detección de tecnologías web y CMS. URL: https://github.com/urbanadventurer/WhatWeb.

[39] Tuhinshubhra. Cmseek - cms detection and exploitation suite. Herramienta de detección de CMS. URL: https://github.com/Tuhinshubhra/CMSeeK.

[40] Gokulapap. Wappalyzer cli. Versión CLI de Wappalyzer para detección de tecnologías web. URL: https://github.com/gokulapap/wappalyzer-cli.

[41] Pyppeteer Contributors. Pyppeteer - headless chrome/chromium automation library. Librería Python para automatizar navegadores en modo headless. URL: https://pypi.org/project/pyppeteer/.

[42] Selenium Contributors. Selenium - official python bindings for selenium webdriver. Librería utilizada para la automatización de navegadores web. URL: https://pypi.org/project/selenium/.

[43] epi052. Feroxbuster - a fast, simple, recursive content discovery tool written in rust. Herramienta para enumeración de contenido web. URL: https://github.com/epi052/feroxbuster.

[44] OJ Reeves. Gobuster. Herramienta para enumeración de contenido web. URL: https://github.com/OJ/gobuster.

[45] Mauro Soria. dirsearch - web path discovery. Herramienta para enumeración de contenido web. URL: https://github.com/maurosoria/dirsearch.

[46] FFUF Contributors. Ffuf. Herramienta para enumeración de contenido web. URL: https://github.com/ffuf/ffuf.

[47] WPScan Team. Wpscan - wordpress security scanner. Herramienta especializada en el análisis de vulnerabilidades y enumeración en sitios WordPress. URL: https://wpscan.com/.

[48] Raiola Networks. Xmlrpc.php de wordpress ¿que es y como protegerlo? Artículo sobre XMLRPC.PHP de WordPress. URL: https://raiolanetworks.com/blog/xmlrpc-php-de-wordpress-que-es-y-como-protegerlo/.

[49] OWASP Foundation. Owasp joomscan - joomla vulnerability scanner. Herramienta especializada en el escaneo de sitios Joomla. URL: https://github.com/OWASP/joomscan.

[50] Python Software Foundation. ftplib — ftp protocol client. Librería de Python para interactuar con servidores FTP. URL: https://docs.python.org/es/3/library/ftplib.html.

[51] Exploit Database. Searchsploit - the manual. Herramienta de línea de comandos para buscar exploits públicos. URL: https://www.exploit-db.com/searchsploit.

[52] Van Hauser. Hydra. Herramienta de fuerza bruta para servicios de red. URL: https://github.com/vanhauser-thc/thc-hydra.

[53] Paramiko Organization. Paramiko - a python implementation of sshv2. Librería Python para la implementación del protocolo SSHv2. URL: https://www.paramiko.org/.

[54] ReportLab Inc. Reportlab. Librería de Python para la generación de documentos PDF. URL: https://docs.reportlab.com/.

[55] Fpdf2 Contributors. fpdf2 - simple and fast pdf generation for python. Librería de Python para la generación de documentos PDF. URL: https://pypi.org/project/fpdf2/.

[56] Google. Gemini api. Documentación oficial de la API Gemini. URL: https://ai.google.dev/gemini-api/docs/.

[57] Mistral. Mistral api. Documentación oficial de la API de Mistral. URL: https://docs.mistral.ai/.

[58] OpenAI. Openai api. Documentación oficial de la API de OpenAI. URL: https://platform.openai.com/docs/.

[59] DeepSeek. Deepseek api. Documentación oficial de la API de DeepSeek. URL: https://api-docs.deepseek.com/.

[60] Wikipedia contributors. Document object model, 2024. Definición del término "Document Object Model (DOM)". URL: https://es.wikipedia.org/wiki/Document_Object_Model.

[61] Matthew Kosinski (IBM). ¿qué es spear phishing?, 2024. Artículo sobre el spear phishing. URL: https://www.ibm.com/es-es/think/topics/spear-phishing.